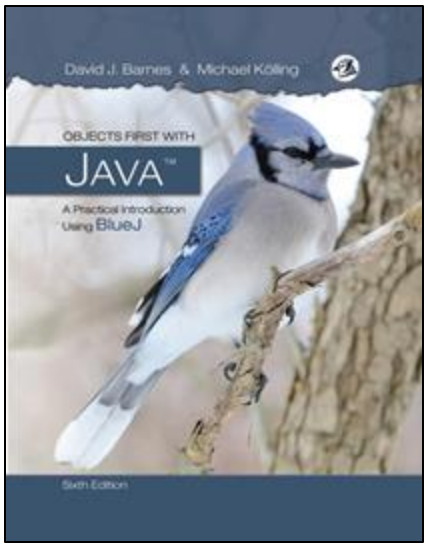
Unidad 4

Agrupando y Ordenando Objetos



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson de los capítulos 4 y 7 del libro "Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition" escrito por David J. Barnes y Michael Kolling" © 2017

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Conceptos fundamentales

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Los Requisitos para Agrupar Objetos

- Muchas aplicaciones usan colecciones de objetos:
  - Agendas de contactos.
  - Catálogos de biblioteca.
  - Gestión de expedientes de estudiantes.
  - Organizador musical.
- El número de ítems que hay que almacenar suele cambiar.
  - Se añaden.
  - Se eliminan.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Colecciones de tamaño fijo

- Algunas veces el tamaño máximo de la colección se puede predeterminar.
- Java proporciona un tipo especial de colección de tamaño fijo: **array**.
- También se diferencian de otras colecciones de Java en que **no solo** pueden almacenar referencias a objetos sino también valores de tipo primitivo (sin hacer autoboxing).

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Características de los Arrays

- Contenedor de tamaño fijo para objetos del mismo tipo
  - Otras colecciones sí son de tamaño variable:
    - **`ArrayList`**, **`HashSet`**, **`HashMap`**, etc.
- Usan una sintaxis especial.
  - Por razones históricas.
- Objetos sin métodos.
  - Los métodos son proporcionados por otras clases; p.ej., **`java.util.Arrays`**.
  - Métodos que son estáticos*.

* Qué significa estático lo veremos al detalle en la Unidad sobre "Comportamientos más avanzados"

- https://docs.oracle.com/javase/specs/jls/se11/html/jls-10.html

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Símil de Arrays en el mundo real?

- La idea de un array es similar a la de una estantería de libros
- Cada libro en el estante está etiquetado con un número
- El número de libros que caben en la estantería está predefinido por el tamaño de la estantería
  - La estantería no se puede estirar ni es infinita



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “Weblog-Analyzer” (Sample Code Unit 4)

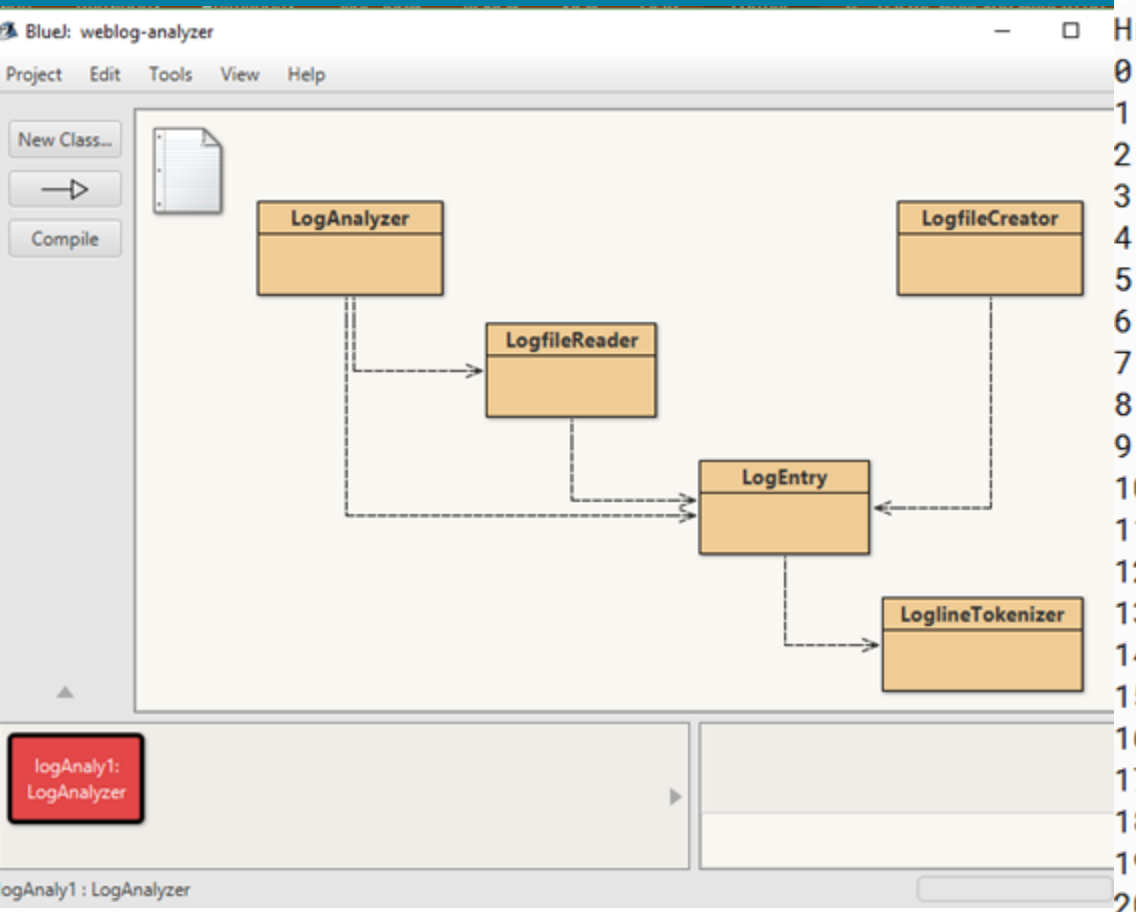
Los servidores Web suelen almacenar registros de cada acceso para realizar análisis:

- Páginas más populares.
- Los períodos de más ocupación.
- Cantidad de datos proporcionada.
- Enlaces rotos.

- Este Proyecto analiza los accesos cada hora ->
  ¡¡Hay un número fijo de horas al día!!

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “Weblog-Analyzer” (Sample Code Unit 4)



BlueJ: Terminal Window - weblog-analyzer

Options

```
LogAnalyzer logAnaly1 = new LogAnalyzer();
logAnaly1.analyzeHourlyData();
logAnaly1.printHourlyCounts();
Hr: Count
0: 149
1: 149
2: 148
3: 109
4: 92
5: 177
6: 185
7: 142
8: 91
9: 85
10: 227
11: 142
12: 114
13: 164
14: 227
15: 185
16: 167
17: 198
18: 237
19: 172
20: 142
21: 113
22: 168
23: 166
```

Pearson

Copyright © 2017, 2012, 2009

Demo de Proyecto “Weblog-Analyzer” (Sample Code Unit 4)

Analiza las entradas desde un log llamado weblog.txt

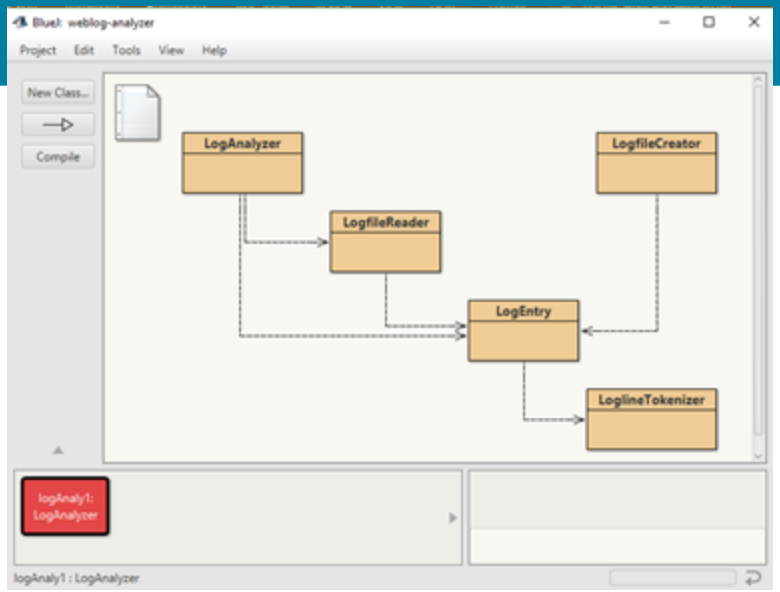
El servidor web se supone que almacena una hora de acceso del tipo

7 de junio de 2011 a las 3:45am

en el formato:

año mes día hora minuto

2011 06 07 03 45



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

El Proyecto Weblog-Analyzer

- El analizador actual está limitado a proporcionar un Contador del número de accesos en cada franja horaria (0 - 23)
- La clase LogAnalyzer contiene el campo:

```
private int[] hourCounts;
```

hourCounts es de tipo array de enteros

- *int* se considera el tipo base del array
- Diferencias entre declaraciones:

```
int hourCounts;// variable sencilla de tipo int
int[] hourCounts;//variable de tipo array de int
```

!!Solo se pueden añadir al array datos de tipo *int*!!

on Education, Inc. All Rights Reserved

Creando un objeto Array

```
public class LogAnalyzer
{
    private int[] hourCounts;
    private LogfileReader reader;

    public LogAnalyzer()
    {
        hourCounts = new int[24];
        reader = new LogfileReader();
    }
    ...
}
```

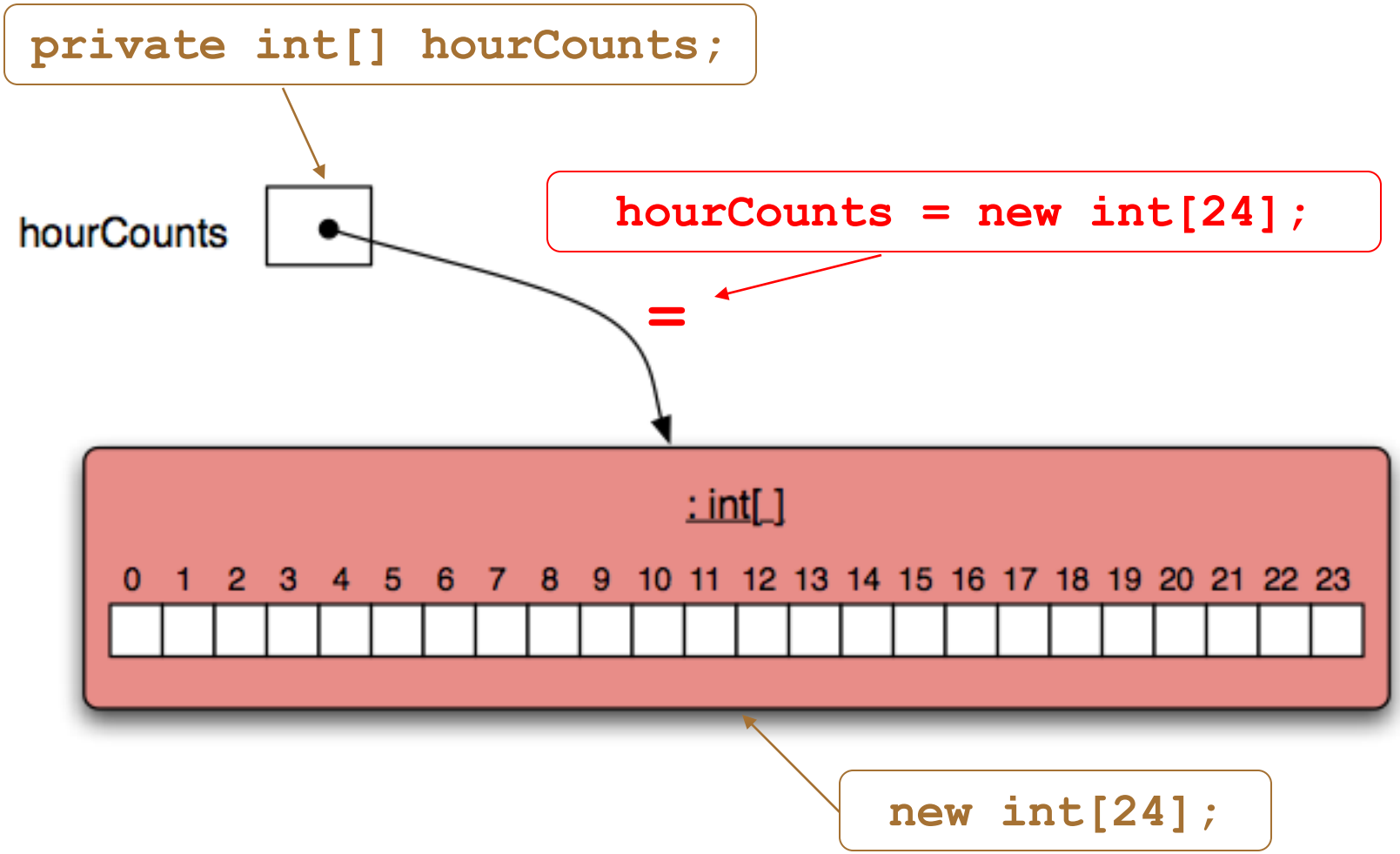
Declaración de variable *Array*
¡¡NO contiene el tamaño!!

Creación del *Array*:
- reserva memoria para un *Array* de 24 *int*
- **Inicializa** todos los elementos del array al valor por defecto del tipo

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

El Array hourCounts



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen de el ejemplo *LogAnalyzer*

- El campo hourCounts es necesario para almacenar el análisis de los datos de acceso

```
hourCounts = new int[24];
```

- El constructor crea un objeto array de 24 enteros para el campo hourCounts




```
hourCounts.length  // igual a 24
```

- Cada uno de los 24 enteros representa el número de accesos hechos dentro de una hora en particular

```
hourCounts[hour]++;
```

  - Un mayor valor entero significa que hubo más accesos en esa franja horaria.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Usando un Array

- Se usa notación de **Corchetes** para **acceder** un elemento de un array mediante indexación: `hourCounts[...]`
- Los elementos se usan como variables ordinarias.
- Como destino de una asignación: `hourCounts[hour] = ...;`
- En una expresión:

```
hourCounts[hour]++;
if(hourCounts[hour] > 0) ...
```

- Los índices válidos dependen del tamaño (length) del array y su rango es [0 … (length-1)]

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Usando un Array

```
private int[] hourCounts;
private String[] names;

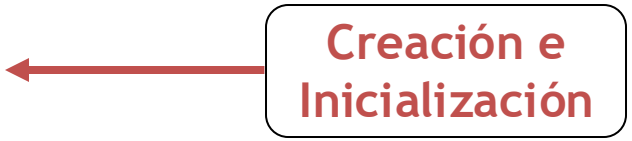
...

hourCounts = new int[24];
names = new String[10];

...

hourCounts[i] = 0;
hourCounts[i]++;
System.out.println(hourCounts[i]);
```







Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Usando un Array

```
1  String[] cases =      new    String[5];
2  cases[0] = "Meeting Eike";
3  cases[1] = "Potsdam Underground";
4  cases[2] = "A Strange Phone";
5  cases[3] = "Hijacked: Paco the Parrot";
6  cases[4] = "The Museum Mystery”
7   System. out.println(cases[0])
8   System. out.println(cases[4])
```

```
> Meeting Eike
  The Museum Mystery
```

- En Java los Arrays no pueden cambiar de tamaño
  - No se puede añadir más elementos de forma dinámica

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Usando un Array

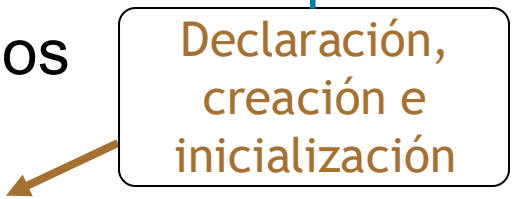
```
1  String[] cases =      new    String[5];
2  cases[0] = "Meeting Eike";
3  cases[1] = "Potsdam Underground";
4  cases[2] = "A Strange Phone";
5  cases[3] = "Hijacked: Paco the Parrot";
6  cases[4] = "The Museum Mystery”
7   System. out.println(cases[0])
8   System. out.println(cases[4])
9   System. out.println(cases[5])
```

```
>_ Meeting Eike
   The Museum Mystery
   Exception in thread "main"
   java.lang.ArrayIndexOutOfBoundsException: 5 ...
```

- Se produce una **excepción** al agregar o acceder a un elemento en una posición que excede el tamaño del Array

Arrays Literales

- Se puede declarar, crear e inicializar al mismo tiempo un Array con otros valores que no sean los predeterminados




```
private int[ ] numbers = {3, 15, 4, 5, 1};
```

- El tamaño se infiere a partir de los datos.

- Esta forma solo puede usarse en el momento de declarar el array, NUNCA de esta forma:

```
numbers = { 3, 15, 4, 5, 1 };
```

XXXXX

- Para hacer un uso similar se requiere **new**:

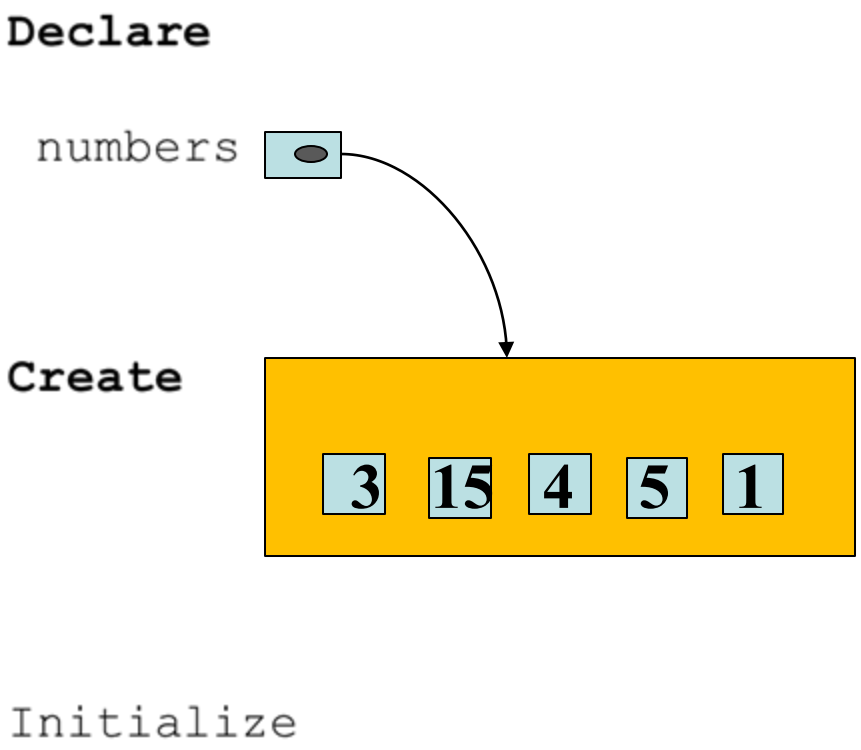
```
numbers = new int[ ] { 3, 15, 4, 5, 1};
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Arrays Literales

```
private int[ ] numbers = { 3, 15, 4, 5, 1 };
```

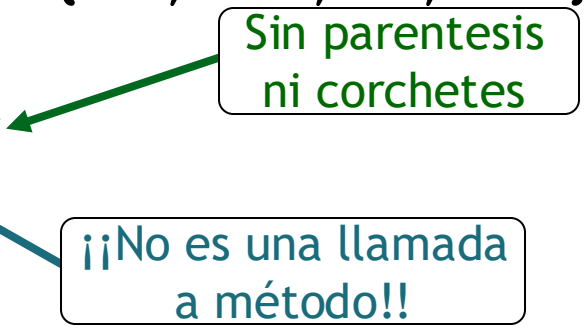


Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tamaño (length) del Array

```
private int[] numbers = { 3, 15, 4, 5 };

int n = numbers.length;
```



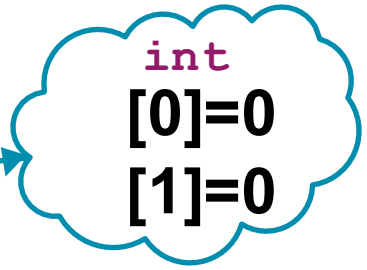
- **`length`** es un **campo** del objeto **Array**.
  - No confundir con **`length()`**, que es un método de **String**
  - No confundir con **`size()`**, que es un método de algunas colecciones como **ArrayList**
- Su valor no puede ser cambiado – '**tamaño fijo**'.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencias en Arrays de tipos primitivos

Dirección	Identificador	Valor
#0		null
#1	nums	
#2		
#3		
#4		
#5		

```
1 int[] nums = new int[2];
```



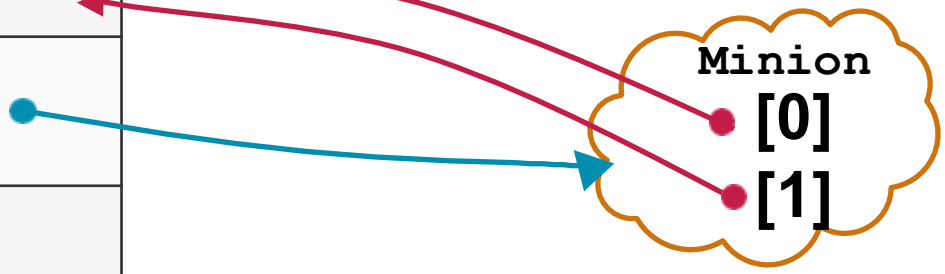
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencias en Arrays de tipo objeto

Dirección	Identificador	Valor
#0		null
#1	minions	
#2		
#3		
#4		
#5		

```
1 Minion[] minions = new Minion[2];
```



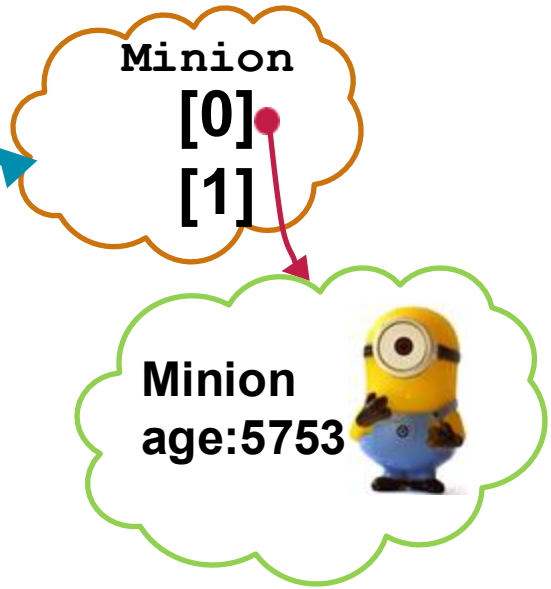
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencias en Arrays de tipo objeto

Dirección	Identificador	Valor
#0		null
#1	minions	
#2		
#3		
#4		
#5		

```
1 Minion[] minions = new Minion[2];
2 minions[0] = new Minion(5753);
```



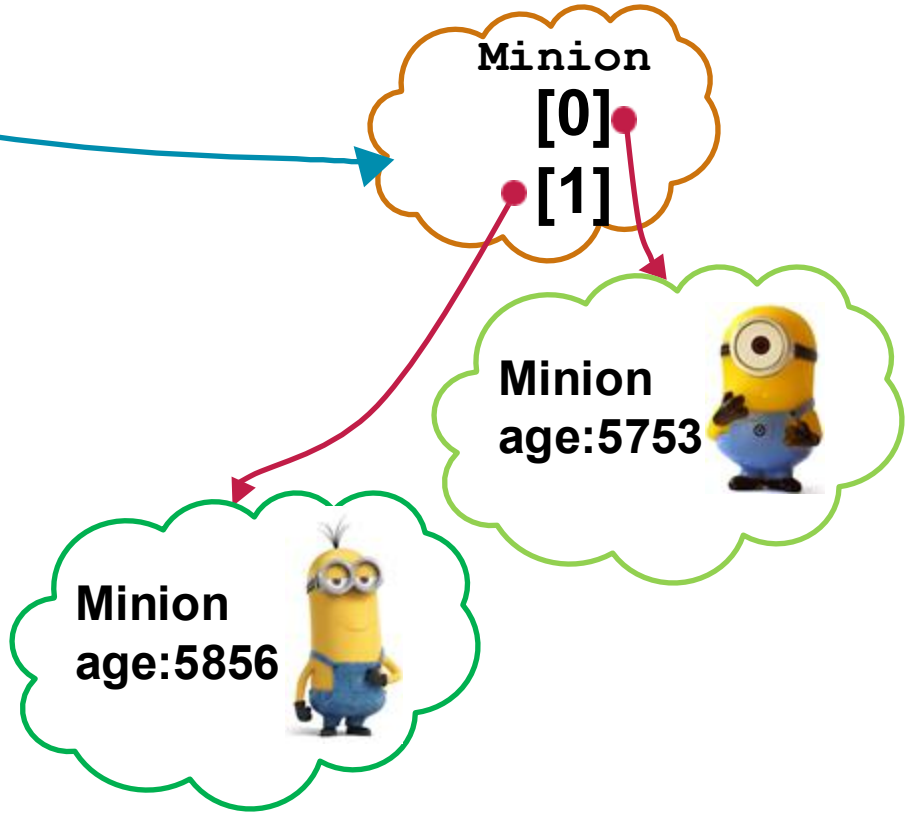
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencias en Arrays de tipo objeto

Dirección	Identificador	Valor
#0		null
#1	minions	
#2		
#3		
#4		
#5		

```
1 Minion[] minions = new Minion[2];
2 minions[0] = new Minion(5753);
3 minions[1] = new Minion(5856);
```



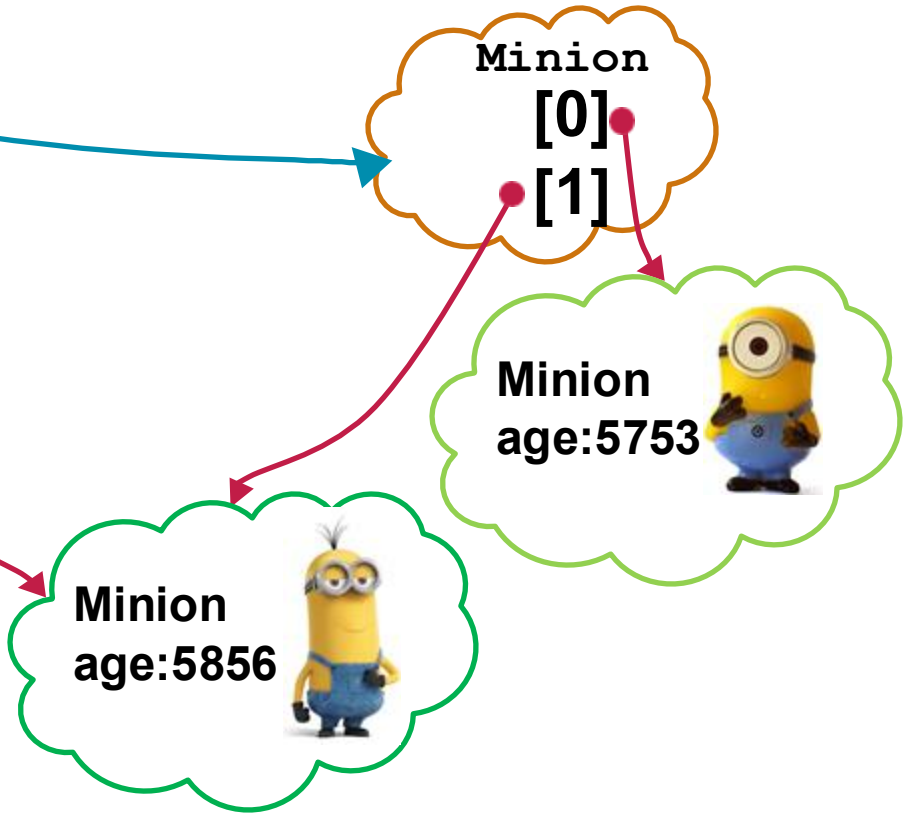
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencias en Arrays de tipo objeto

Dirección	Identificador	Valor
#0		null
#1	minions	
#2	kevin	
#3		
#4		
#5		

```
1 Minion[] minions = new Minion[2];
2 minions[0] = new Minion(5753);
3 minions[1] = new Minion(5856);
4 Minion kevin = minions[1];
```



Pearson

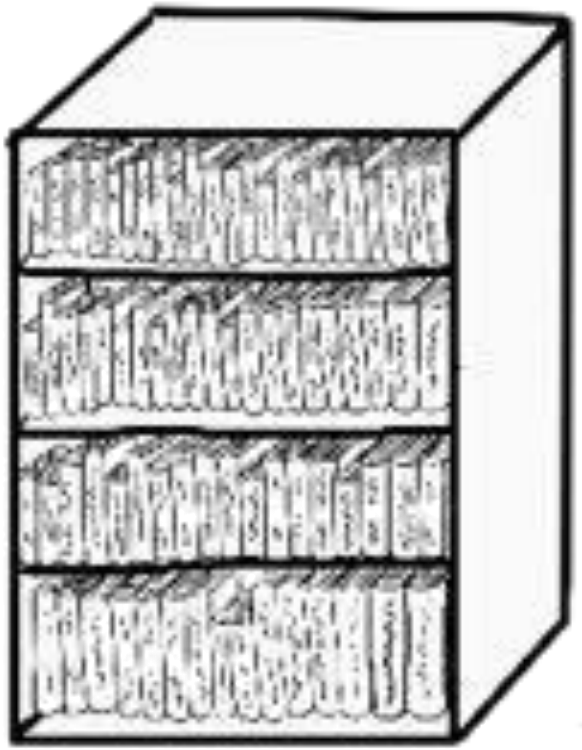
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Símil de Arrays en el mundo real

- ¿Qué pasa si queremos añadir más libros de los que caben en la estantería?
- ¿Qué pasa con la numeración de cada libros si quitamos un libro del medio?
- ¿Cómo sabemos cuántos libros tenemos realmente en la estantería?

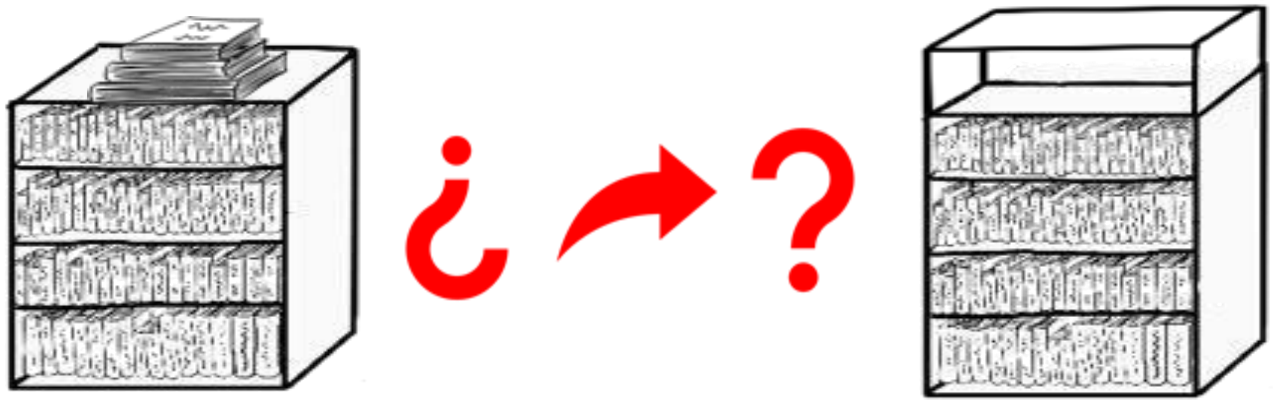


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Símil de Arrays en el mundo real

- Los Arrays NO PUEDEN cambiar de tamaño
- Si queremos añadir un elemento en la posición inicial, tenemos que desplazar todos los elementos posteriores de forma manual
  - ¿Qué le pasaría al último elemento?



¡¡Necesitamos algo más aparte de Arrays!!

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Colecciones

- Agrupar objetos es un requisito recurrente.
  - El paquete **java.util** contiene múltiples clases para hacerlo llamadas **Colecciones**.
  - Son clases de la Biblioteca Java ”listas para ser usadas”.
  - El término **Colecciones** se puede considerar un término “paraguas” para agrupar las diferentes estructuras específicas disponibles en Java



    - Pueden contener elementos (objetos)
    - Algunas pueden mantenerlos en un cierto orden
    - Ofrecen acceso a los elementos:
      - Eficientemente
      - Basándose en características específicas (posición, clave)
    - No representan nada por sí mismas, son meta-estructuras

Colecciones

- Todas las **Colecciones** permiten:
  - Añadir elementos (add())
  - Eliminar elementos (remove())
  - Comprobar si se tiene un elemento (contains())
  - Recuperar la cantidad real de elementos que contiene (size())
  - Comprobar si están vacías (isEmpty())
- Son estructuras específicas adecuadas para casos de uso específicos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

ArrayList

- La clase de biblioteca **ArrayList** es una de las **Colecciones** más utilizadas:
  - Es una lista implementada con Arrays:
    - Simula un array "redimensionable"
    - Crece según nuestras necesidades
  - Mantiene los objetos en orden de inserción (por defecto igual que en una lista)
  - Permite el acceso directo a los objetos en cualquier posición
  - Maneja el acceso general a los objetos
    - Los detalles de cómo lo hace están ocultos.
      - ¿Importa?¿No saberlo evita que lo usemos?

Pearson

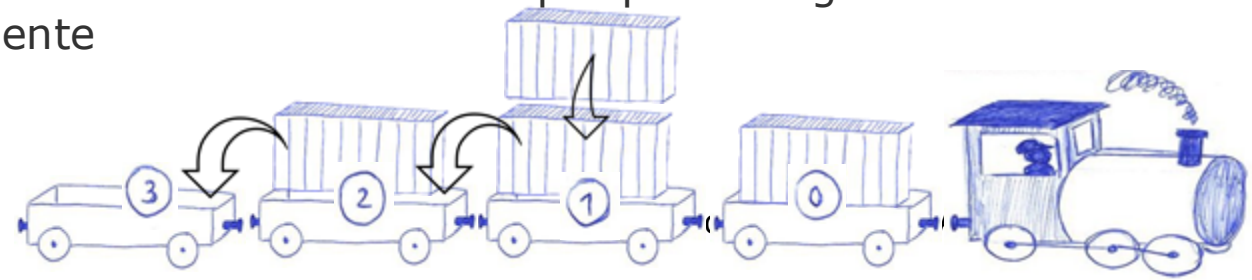
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/ArrayList.html#method.summary

Métodos clave de ArrayList

- La clase ArrayList implementa una lista de funcionalidades con métodos para las siguientes operaciones:
  - Añadir elemento: add(item)
  - Añadir elemento en posición concreta*: add(index, item)
  - Eliminar elemento: remove(item)
  - Eliminar elemento en posición concreta: remove(index)
  - Recuperar elemento en posición concreta: get(index)
  - Consultar el tamaño actual: size()

*: Añadir un elemento intermedio es más costoso porque los siguientes elementos se desplazarán automáticamente



Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales

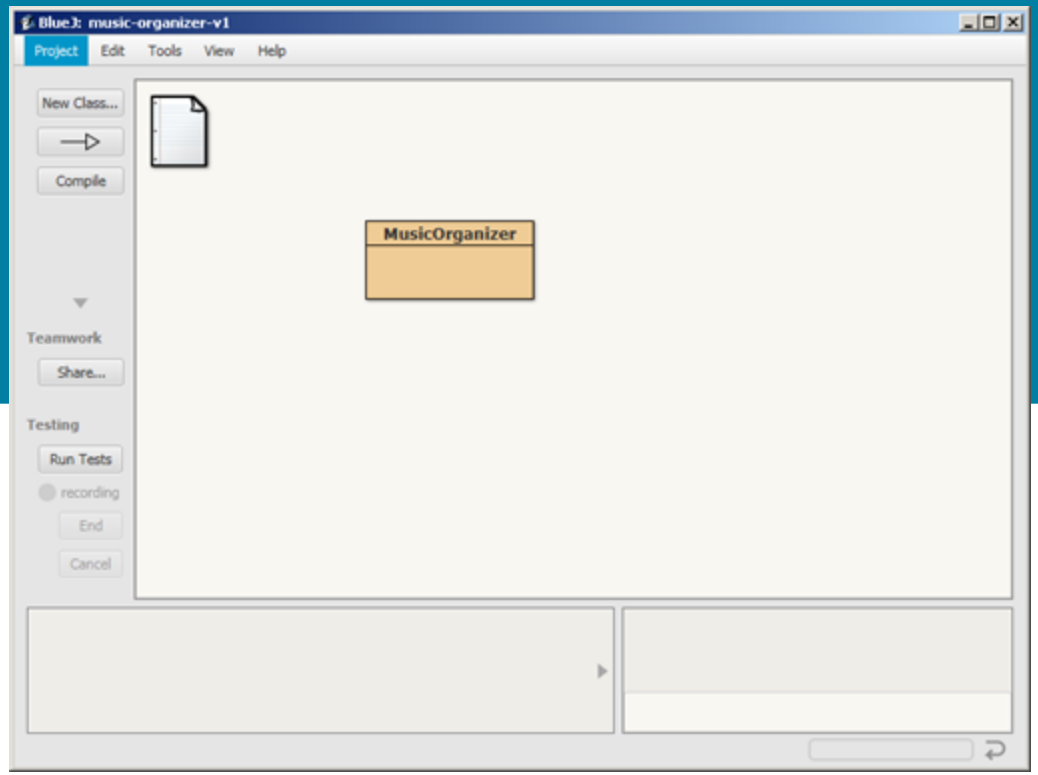
- Se pueden añadir archivos con una sola pista (canción).
- No hay un límite predefinido para el número de archivos/pistas.
- Indicará cuántos nombres de archivo se almacenan en la colección.
- Enumerará los nombres de los archivos individuales.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales



- Explorar el proyecto **music-organizer-v1**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales

- COLECCIÓN
  - Contiene un grupo de canciones
- ITEMS
  - Canciones se almacenan solo con su nombre de archivo
  - No hay un máximo predefinido de canciones

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales

- OPERACIONES
  - Se pueden añadir nombres de ficheros
  - Se pueden eliminar nombres de ficheros
  - Cuántos nombres están almacenados (tamaño)
  - Seleccionar un nombre de fichero del grupo

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales

- CLASE
- MusicOrganizer conteniendo varios nombres de ficheros de canciones
- CAMPOS
- Una estructura dinámica (**ArrayList**) para un número variable de nombres de ficheros de canciones (String)

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales

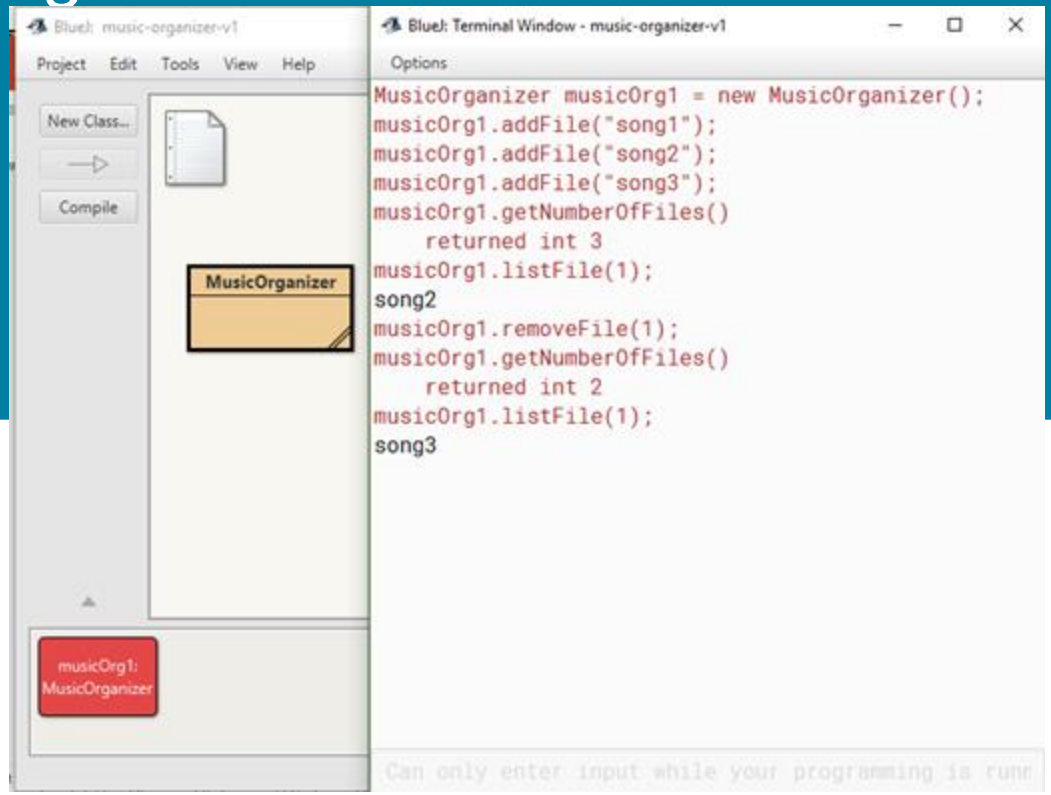
- MÉTODOS
  - addFile
  - removeFile
  - getNumberOfFiles
  - listFile

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “music-organizer-v1” (Sample Code Unit 4)

- Organizador de ficheros musicales



- Prueba el proyecto **music-organizer-v1**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La clase Music-Organizer

```
import java.util.ArrayList;
 /** */
public class MusicOrganizer
{
    // Storage for an arbitrary number of file names.
    private ArrayList<String> files;

    /**
     * Perform any initialization required for the
     * organizer.
     */
    public MusicOrganizer()
    {
        files = new ArrayList<String>();
    }

    ...
}
```

Una clase dentro del paquete **java.util**

El tipo de dato pasado a **ArrayList**

No se necesitan argumentos para el constructor de este **Arraylist<String>**

Pearson

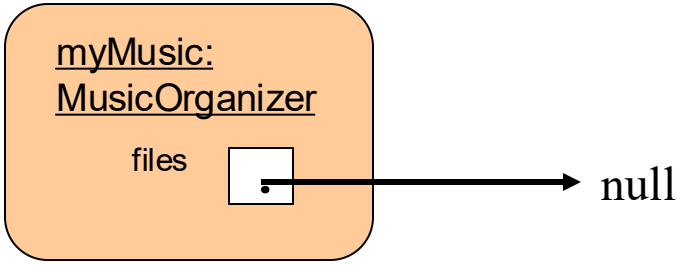
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Uso de ArrayList en Music-Organizer

- Especificamos:
  - El tipo de colección: **`ArrayList`**
  - El tipo de objeto que contiene: **`<String>`**

```
private ArrayList<String> files;
```

  - Solo 1 campo llamado *files* que es definido para toda la clase



- Decimos, “ArrayList de String”.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Clases Genéricas (1 de 2)

- Las **colecciones** son conocidas como tipos **parametrizados** o **genéricos**.
- `ArrayList<parameter-type>` implementa funcionalidad de una lista: **`add`**, **`get`**, **`size`**, etc.
- El tipo de parámetro dice de qué queremos la lista:
  - `ArrayList<Person>`
  - `ArrayList<TicketMachine>`
  - etc.
- Un **ArrayList** puede almacenar cualquier tipo de objeto, pero TODOS los objetos en la lista deben ser del mismo tipo

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Clases Genéricas (2 de 2)

- Podemos usar `ArrayList` con **cualquier** tipo de clase:

```
ArrayList<TicketMachine>
ArrayList<ClockDisplay>
ArrayList<Track>
ArrayList<Person>
```

- Cada una almacenará multiples objetos del tipo específico.

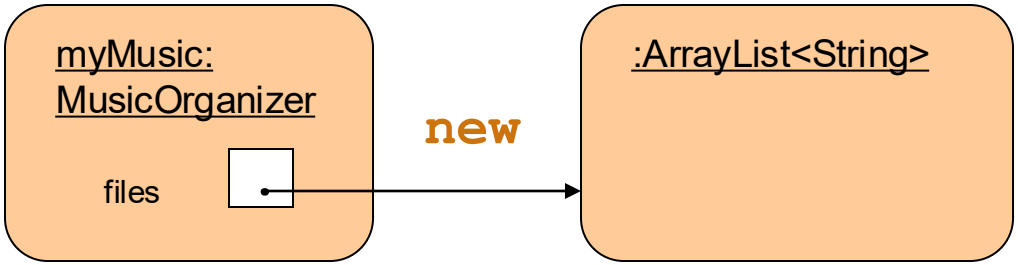
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Creando un objeto Arraylist

```
private ArrayList<String> files;
```

- En versions anteriores a Java 7
  - `files = new ArrayList<String>( );`
- Java 7 introdujo la 'notación diamante'
  - `files = new ArrayList< >( );`
    - El tipo de parámetro puede inferirse de la variable que es asignada.
      - **Es una ventaja que es recomendable usar**



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Usando ArrayList

```
public class MusicOrganizer
{
    private ArrayList<String> files;

    ...

    public void addFile(String filename)
    {
    files.add(filename);
    }

    public int getNumberOfFiles()
    {
    return  files.size();
    }

    ...
}
```

Añadiendo un nuevo fichero

Devolviendo el número de ficheros (por *delegación*)

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Recuperando un objeto del ArrayList

```
public void listFile(int index)
{
    if(index >= 0 && index < files.size()) {
        String filename = files.get(index);
        System.out.println(filename);
    }
    else {
        // This is not a valid index.
    }
}
```

Validación de índice entre [0 … size-1]

Recupera y muestra el nombre del fichero

¿Es necesario? (¿Mensaje de Error?)

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminando un objeto del ArrayList

```
public void removeFile (int index)
{
    if (index >= 0 && index < files.size()) {
        // This is a valid file.
            files.remove(index);
    }
}
```

Validación del índice

Eliminación del ítem

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

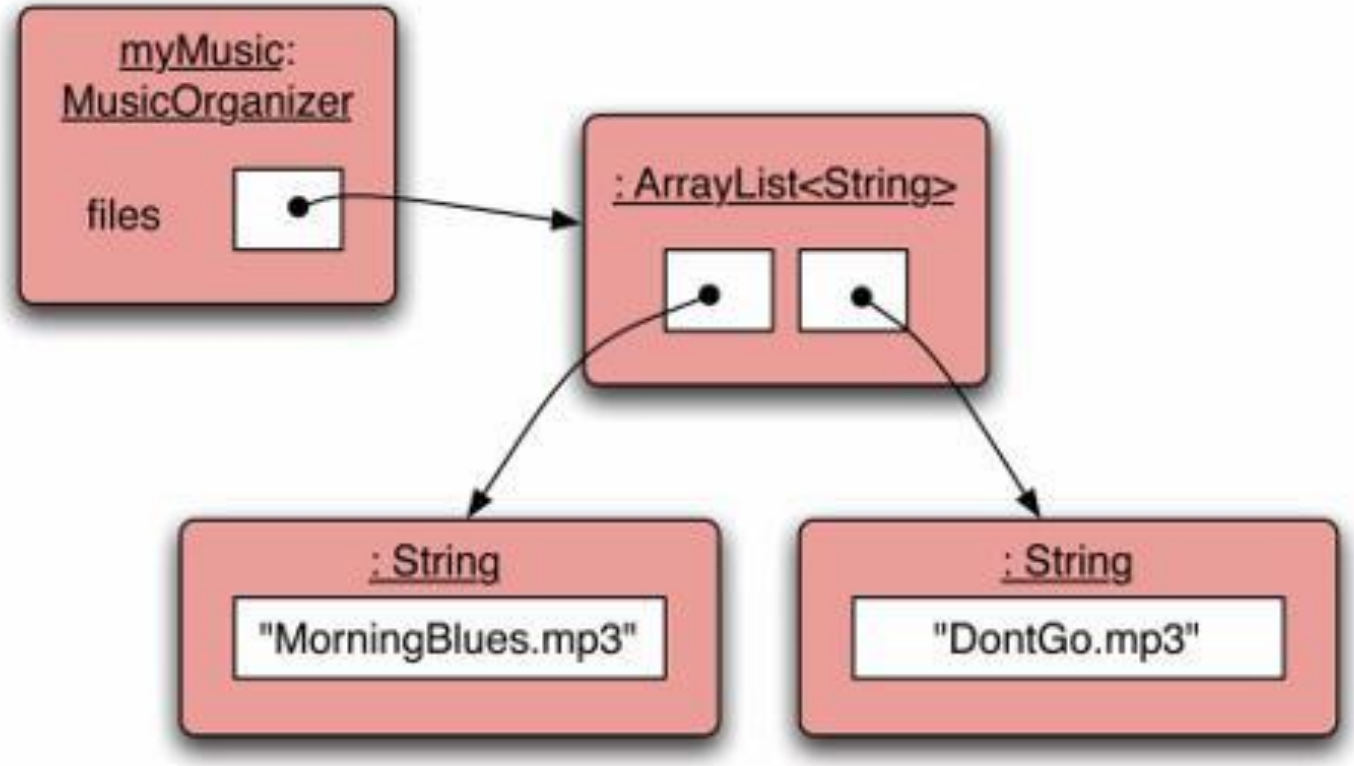
La Utilidad General de los Índices

- Valores de índice:
  - Empiezan en 0
  - Se numeran de forma secuencial
  - No hay huecos entre objetos consecutivos
- Usar enteros para los índices tiene una utilidad general:
  - 'siguiente' es: `index + 1`
  - 'previo' es: `index - 1`
  - 'último' es: `list.size() - 1`
  - 'los tres primeros' es: los ítems con índices `0, 1, 2`
- Podríamos pensar también en acceder a los ítem secuencialmente: `0, 1, 2, …`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Estructura de Objetos con ArrayList



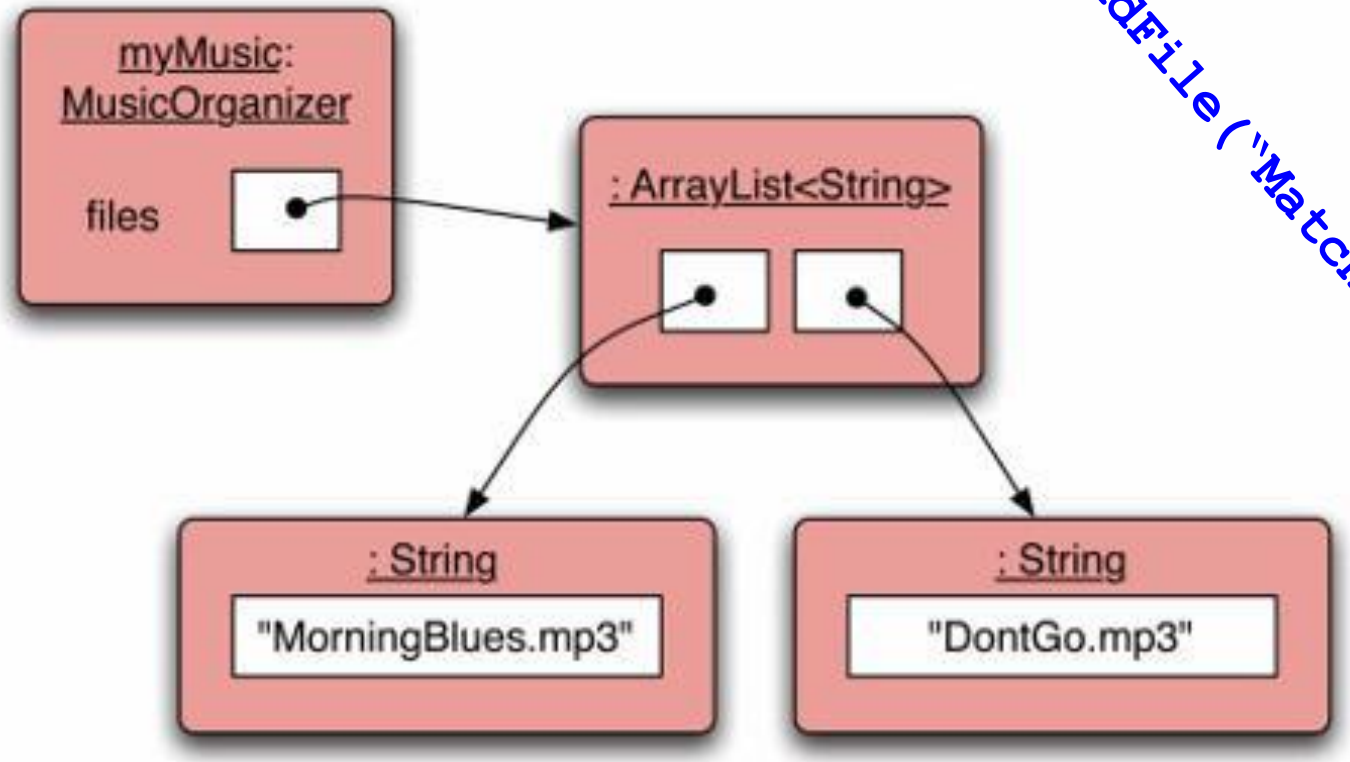
- Sólo un único campo que almacena un objeto de tipo ArrayList<String>
- Todo el trabajo para acceder y gestionar los datos se hace en el ArrayList
- Beneficios de la abstracción al no saber los detalles de cómo se hace
- Nos ayuda a evitar duplicados de información y comportamiento

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Añadiendo un Tercer Fichero

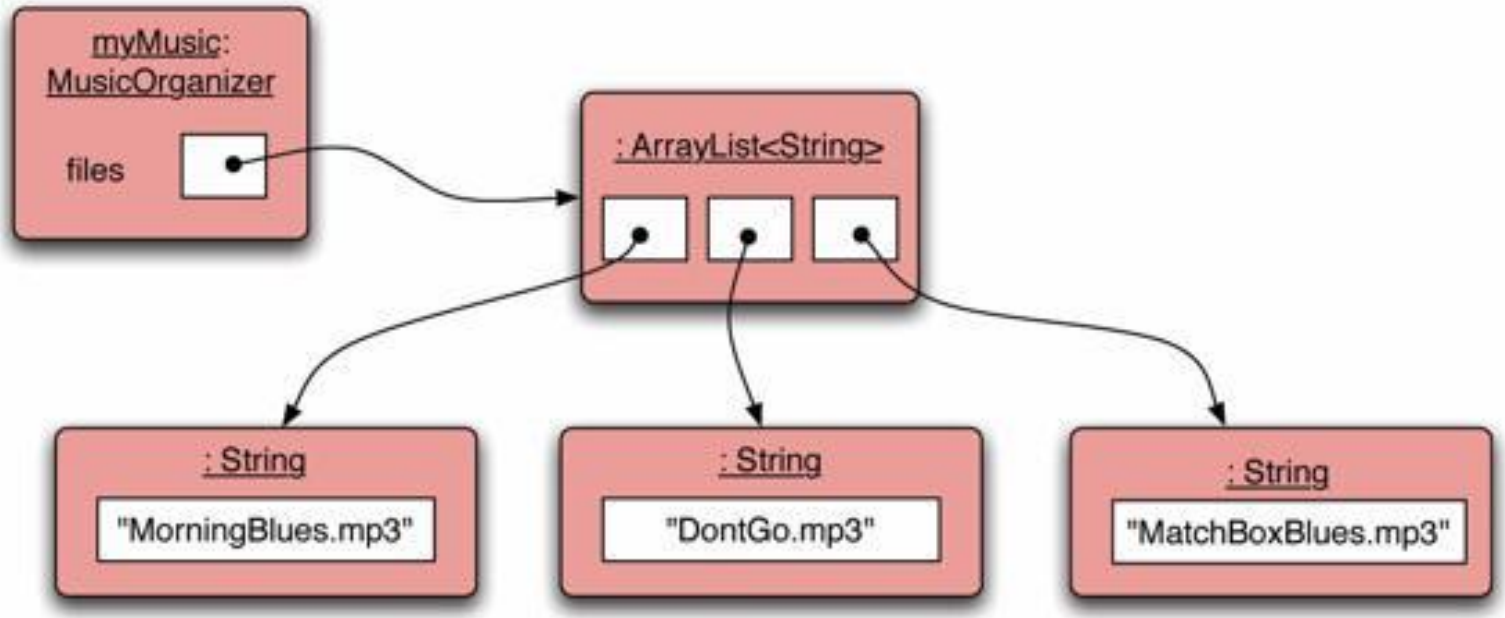
```
myMusic.addFile("MatchBoxBlues.mp3");
```



- Capacidad dinámica para incrementar y/o decrementar cuando se necesita usando los métodos add( ) y remove( )

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

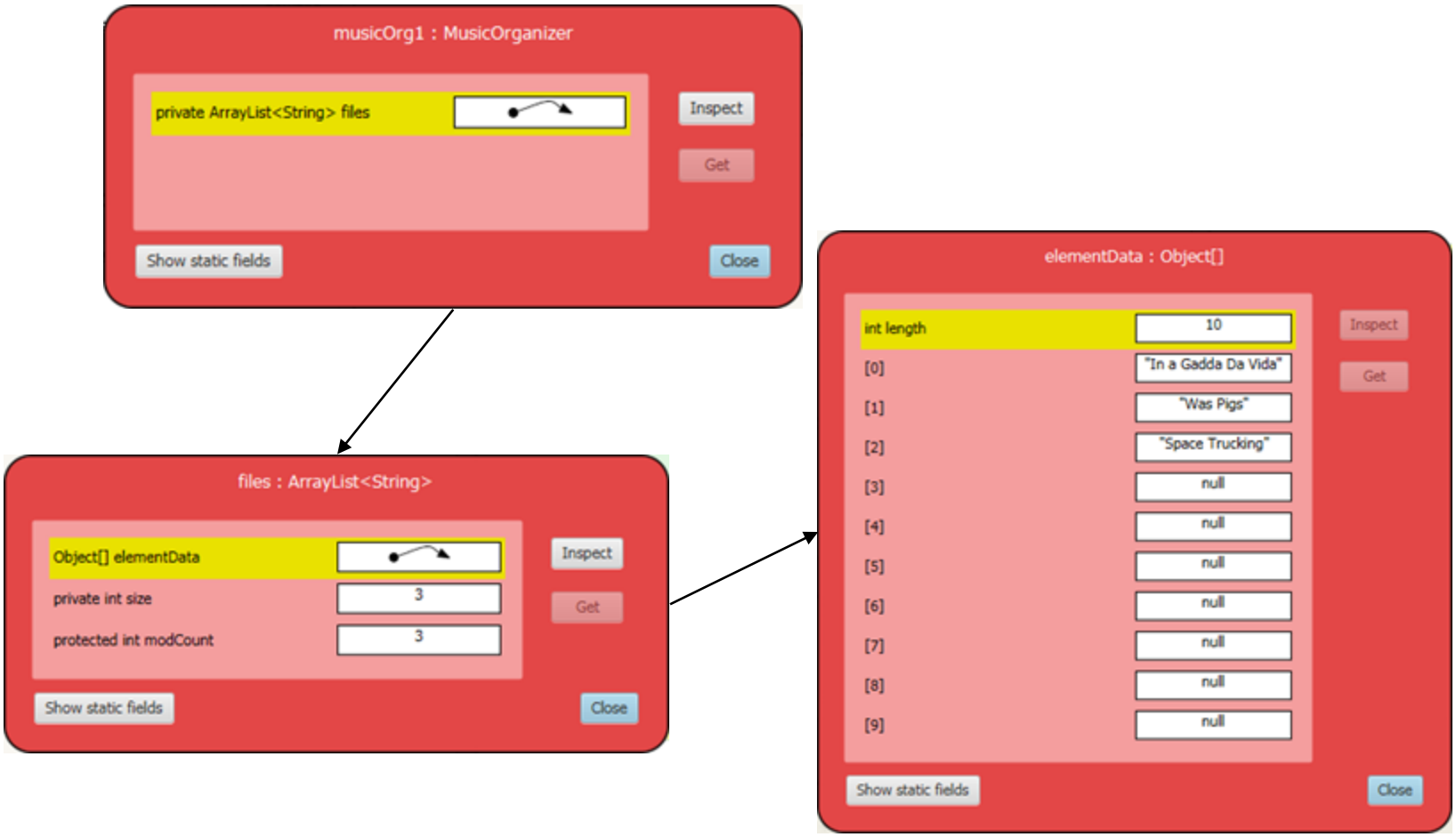
Añadiendo un Tercer Fichero: resultado



- Mantiene un recuento interno del número de ítems que puede devolvernos el método size( )
- Mantiene los artículos en el orden en que fueron insertados con cada nuevo ítem añadido al final de la lista
- Cuando se elimina un ítem, todos los ítems detrás del eliminado se desplazan para llenar el espacio del ítem eliminado.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

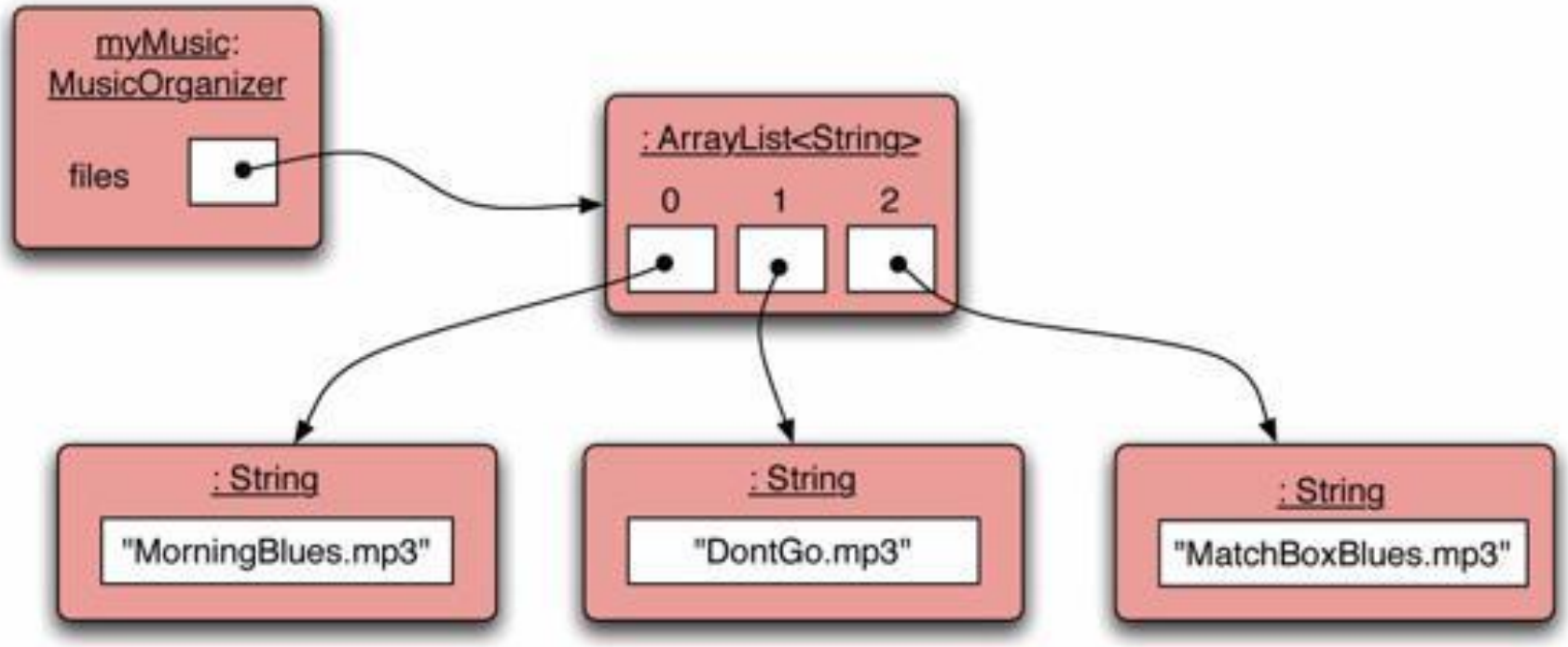
Estructura de Objetos con ArrayList



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

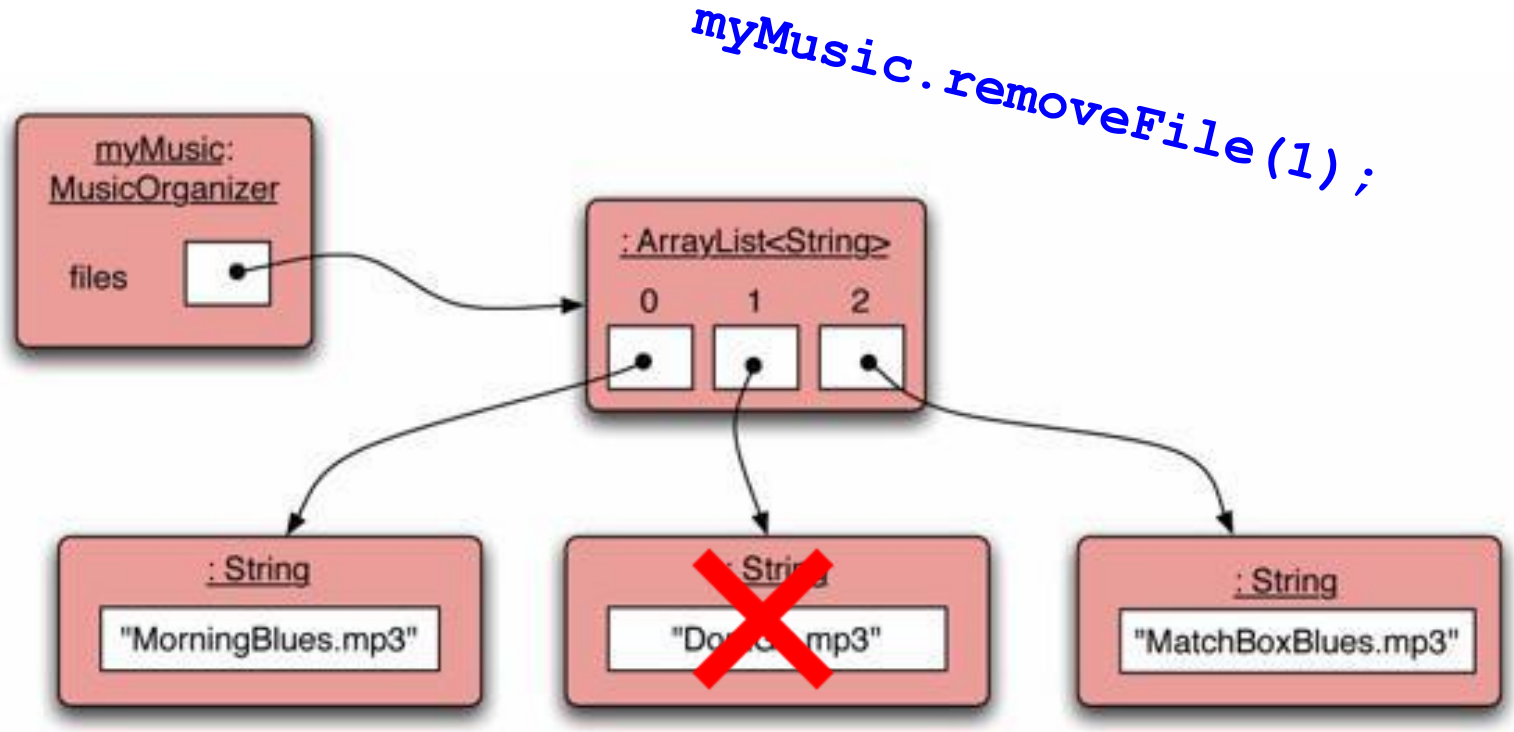
Numeración del índice



- Numeración implícita que comienza con índice 0
- El último ítem tiene como índice size() - 1
- Por tanto, los índices válidos estarían entre [0 . . . size() - 1]

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

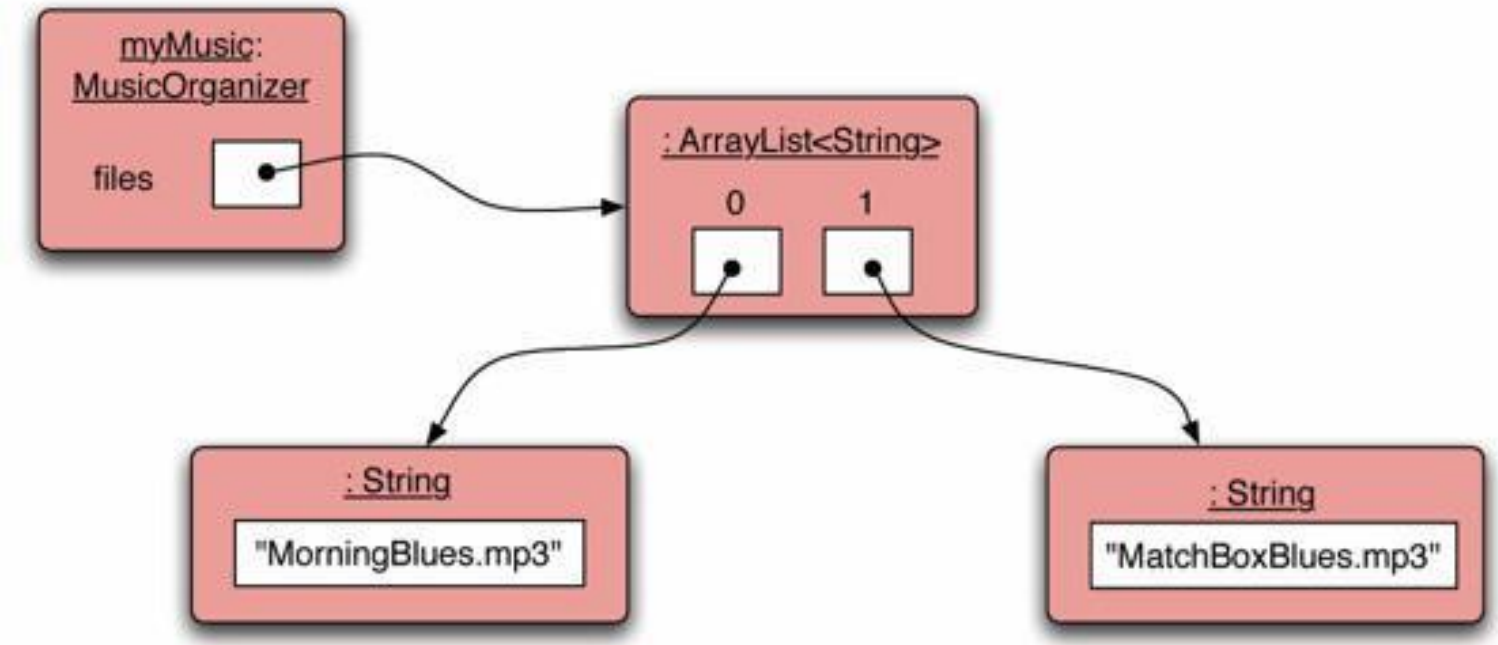
Eliminar puede afectar a la Numeración



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminar puede afectar a la Numeración



- El proceso de eliminación puede cambiar el índice de otros objetos de la lista
- **La colección mueve todos los ítems subsiguientes una posición para llenar el hueco**
- Los índices de los ítems anteriores al eliminado NO SE CAMBIAN
- Los índices de los ítems posteriores al eliminado SE DECREMENTAN en 1
- El mismo "desplazamiento" de los ítems también puede ocurrir si se agregan nuevos ítems en posiciones distintas a la final

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Cómo copiar un *ArrayList* existente

```
ArrayList<Track> copiedList = new ArrayList<Track>(tracks);
```

- Declara una variable *ArrayList* con el mismo tipo de elemento del *ArrayList* original
- Crea un nuevo objeto *ArrayList* (con el mismo tipo de elemento del ArrayList original) para almacenar en él la copia
  - Pasa el *ArrayList* original como parámetro
- La variable ***copiedList*** apunta a la nueva COPIA de la lista original con exactamente el mismo contenido

AVISO:

Solo se tiene UNA instancia de cada objeto –

¡¡pero DOS *ArrayLists* que apuntan a los mismo objetos en el mismo orden!!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Iteración

- A menudo queremos realizar algunas operaciones repetidamente.
  - P.ej, mostrar todos los minions almacenados en un array.
- Muchos lenguajes de programación incluyen la posibilidad de usar **bucles**.
- Java tiene varios tipos de **bucles**.
  - Empezaremos con el **bucle for** "clásico".

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Iteración

- Es el proceso de **repetir** algunas **acciones una y otra vez**.
- Los bucles proporcionan una forma de **controlar cuántas veces** se repiten esas acciones.
- Con una colección a menudo queremos repetir las acciones **exactamente una vez por cada objeto en la colección**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

El bucle for “clásico”

- Hay dos variantes en Java del bucle For:
  - **for** “clásico”.
  - **for-each**
- El **for** “clásico” se usa para iterar un número **fijo** de veces.
- Usa **variable** que cambia una cantidad fija en cada iteración.
- **No podemos salir del bucle antes de que termine**;
  - p,.ej., si solo queremos encontrar la primera coincidencia
  - Realmente, **usando *return***, un bucle for-each puede terminar antes ->**NO DEBE HACERSE NUNCA**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Pseudocódigo de bucle For “clásico”

Forma general del bucle for “clásico”

Cabecera del bucle

```
for(inicialización;condición de continuación; acción posterior)  {
     sentencias a repetir en cada iteración
}
```

```
System.out.println("1: impar");
System.out.println("2: par");
System.out.println("3: impar ");
System.out.println("4: par ");
System.out.println("5: impar ");
System.out.println("6: par ");
System.out.println("7: impar ");
System.out.println("8: par ");
System.out.println("9: impar ");
System.out.println("10: par ");
[...]
System.out.println("99: impar ");
System.out.println("100: even");
```



```
String paridad;
for (int i = 1; i <= 100; i++) {
    if ( i % 2 == 0) {
        paridad = “par";
    }
    else {
        paridad = “impar";
    }
    System.out.println(i + ": " +
paridad);
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Iteración en Arrays

```
for(int hour = 0; hour < hourCounts.length; hour++) {
    System.out.println(hour + ": " + hourCounts[hour]);
}
```

Nota: la variable **hour** no existe al salir del bucle

```
0: 149
1: 149
2: 148
3: 109
4: 92
5: 177
6: 185
7: 142
8: 91
9: 85
10: 227
11: 142
12: 114
13: 164
14: 227
15: 185
16: 167
17: 198
18: 237
19: 172
20: 142
21: 113
22: 168
23: 166
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Bucle For es Flexible

Impresión de múltiplos de 3 por *debajo* de 40

- Inicio en cualquier elemento

```
for(int num = 3; num < 40; num = num + 3) {
    System.out.println(num);
}
```

- Fin en cualquier elemento

```
for(int num = 3; num < 40; num = num + 3) {
    System.out.println(num);
}
```

- Pasos más grandes con incrementos mayores

```
for(int num = 3; num < 40; num = num + 3) {
    System.out.println(num);
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ventajas e inconvenientes del for “clásico”

- PROS
  - puede ser usado con colecciones, sin colecciones o con arrays
  - Flexibilidad para empezar/terminar en un ítem e incrementar la cantidad
  - **capacidad de añadir/eliminar/cambiar el ítem durante el bucle**
  - **acceso a la variable contador del bucle**
  - el incremento se realiza de forma automática después de cada iteración
  - puede usarse en combinación con *Iterators*
- CONS
  - **Iteración definida lo que implica que el número de elementos DEBE conocerse**
  - Acceso a ítems en secuencia [de principio a fin]

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

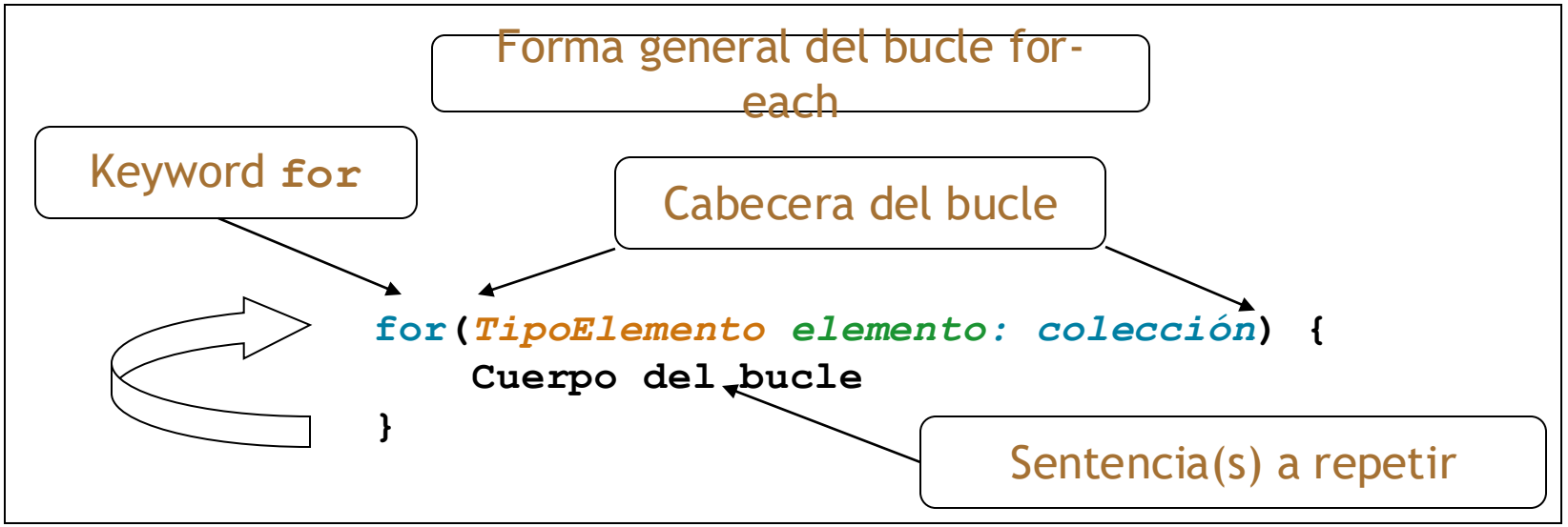
El bucle *for-each*

- Permite iterar sobre la colección al completo.
- Con el bucle for-each **cada** objeto en la colección está disponible **exactamente una vez** en el cuerpo del bucle.
- Pero el bucle for-each **NO** proporciona el índice de la posición del elemento actual
- ¡Es de solo-lectura!
  - No se pueden añadir, borrar o modificar elementos al recorrer la colección

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Pseudo Código del bucle *for-each*



Expresión en pseudo Código de las acciones de un bucle for-each

Para cada *elemento* en *colección*, hacer acciones en Cuerpo del bucle.

**** donde `elemento` es de hecho una declaración de variable de tipo `TipoElemento` y la variable es conocida como *variable del bucle***

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Java

music-organizer-v3

```
/**
 * List all file names in the organizer.
 */
public void listAllFiles()
{
    for(String filename : files) {
        System.out.println(filename);
    }
}
```

Debe ser un objeto de una clase *colección* – como `ArrayList<String>`

Para cada *filename* en *files*, mostrar *filename*

- la keyword for tiene los detalles del bucle entre ( )
- la variable de bucle filename es declarada de tipo String
- el cuerpo del bucle se repite para cada elemento en el ArrayList files
- cada vez, la variable filename mantiene uno de los elementos
- permite acceso al objeto para ese elemento en particular

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Procesamiento Selectivo

- Se pueden anidar sentencias, dando mayor posibilidad de selección a las acciones:

```
public void findFiles(String searchString)
{
    for(String filename : files) {
        if(filename.contains(searchString)) {
            System.out.println(filename);
        }
    }
}
```

contains ofrece un emparejamiento parcial con filename; usa *equals* si quieres un emparejamiento exacto

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Crítica de *for-each*

- Fácil de escribir.
- Termina de forma natural.
- **La colección no puede ser modificada con las acciones**.
- No se proporciona índice.
  - No todas las colecciones se basan en índices.
- **No podemos salir del bucle antes de que termine**;
  - p,.ej., si solo queremos encontrar la primera coincidencia
  - Realmente, **usando *return***, un bucle for-each puede terminar antes ->**NO DEBE HACERSE NUNCA**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Arrays y bucles *for-each*

- ¿Podemos usar *for-each* para acceder CADA elemento en el array sin añadir/eliminar elementos? **Sí**
- Los bucles *for-each* se pueden usar en arrays:

```
for(int value : hourCounts) {
    System.out.println(value);
}
```

- Sin embargo, NO hay contador de índice si se necesita la posición del elemento.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Cuidado con resultado inesperado con *for-each*

- ¡Recuerda!

Los elementos de la Colección o el array accedidos a través de un bucle *for-each* son *copias* de los elementos reales:

```
int[] numbers = { 4, 1, 22, 9, 14, 3, 9};

for (int n : numbers) {
    System.out.println(n);
}
```

☺ **Funciona**



Pero no trates de asignar:

```
for (int n : numbers) {
    n = 2*n;
}
```

☹☹☹

Cambia las *copias* de elementos en **numbers** – ¡no cambia ningún elemento en **numbers**!

☹☹☹

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Revisión de *for-each*

- PROS
  - **fácil de escribir y usar**
  - **accede a TODOS los elementos uno por uno**
  - capaz de cambiar el estado de un elemento
  - **termina automáticamente**
  - permite procesamiento selectivo usando sentencias if-else
  - el cuerpo no tiene límite de sentencias
  - se puede asar con CUALQUIER colección
  - abstrae de los detalles del manejo
- CONS
  - **no proporciona índice**
  - **NO puede parar antes de hacer el recorrido completo**
    - **Recuerda, usar *return NO DEBE HACERSE***
  - **NO se pueden añadir, eliminar o modificar elementos durante el bucle**
  - el acceso es a TODOS los elementos en secuencia [0 a size-1]

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Equivalencia entre For y For-Each

```
public void listAllFiles()
{
     for(String filename : files) {
          System.out.println(filename);
     }
}
```

```
public void listAllFiles()
{
     for(int index = 0;index < files.size(); index++){
          String filename = files.get(index);
          System.out.println(filename);
        }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Iteración definida vs indefinida

- En una iteración definida se sabe previamente el número de iteraciones:
  - for “clásico”: valor máximo de índice / incremento
  - for-each: tamaño del array o colección

- En una iteración indefinida el número de iteraciones no se conoce previamente y puede ser de 0 a infinito

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Las búsquedas son indefinidas

- Supón que estás buscando tus llaves de casa.
- No puedes predecir, **previamente**, en cuantos sitios tendrás que mirar.
- Aunque, puede haber un límite absoluto
  - p.ej., mirar todas las posibles localizaciones.
- Pararás la búsqueda cuando las encuentres.
- Son posibles '**Bucles Infinitos**'.
  - Producido por error o por la naturaleza de la búsqueda.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

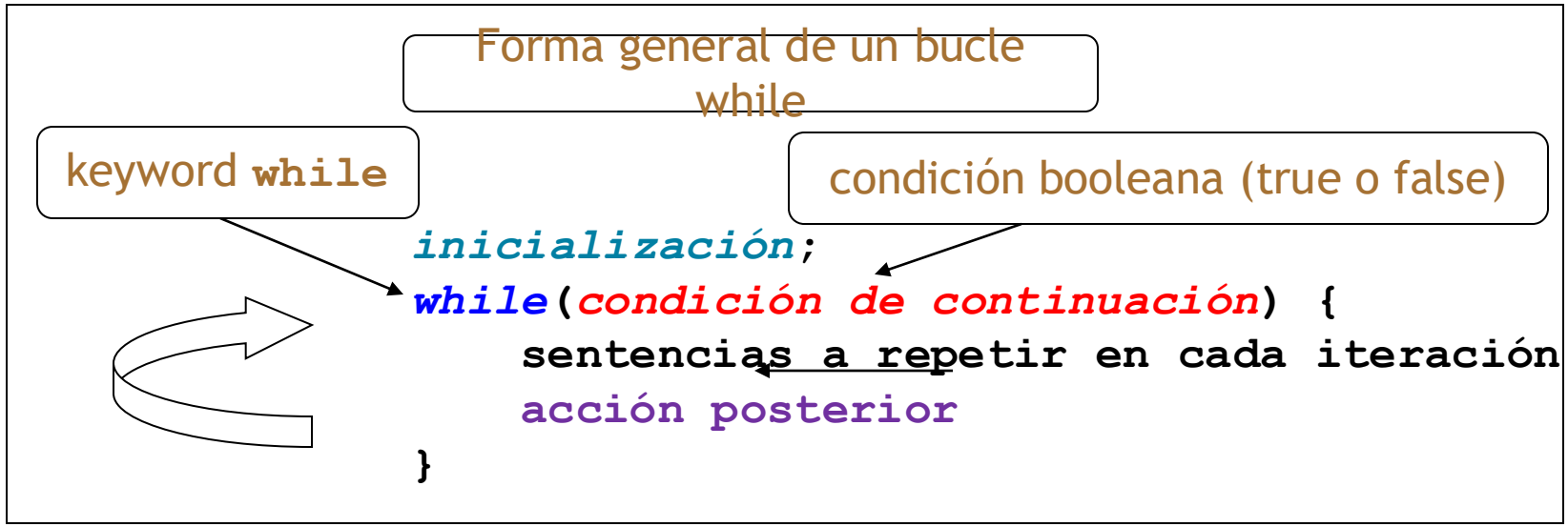
El bucle While

- Es más **flexible** al no estar restringido previamente a:
  - realizar un número conocido de iteraciones (for)
  - recorrer toda la colección o el array (for-each)
- Se usa una condición **booleana** para decidir si se sigue iterando o no.
- Puede que NO haya necesidad de buscar hasta el final
- Se puede usar no solo con colecciones y arrays.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Pseudocódigo de bucle While



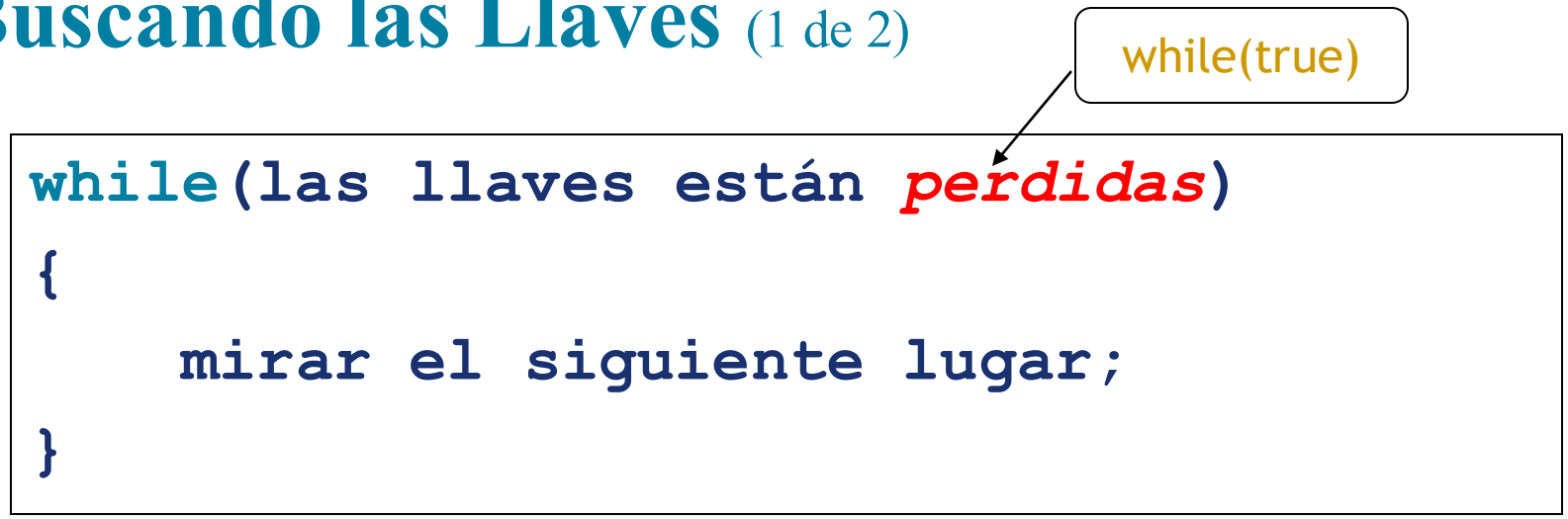
Equivalencia en forma de bucle for "clásico"

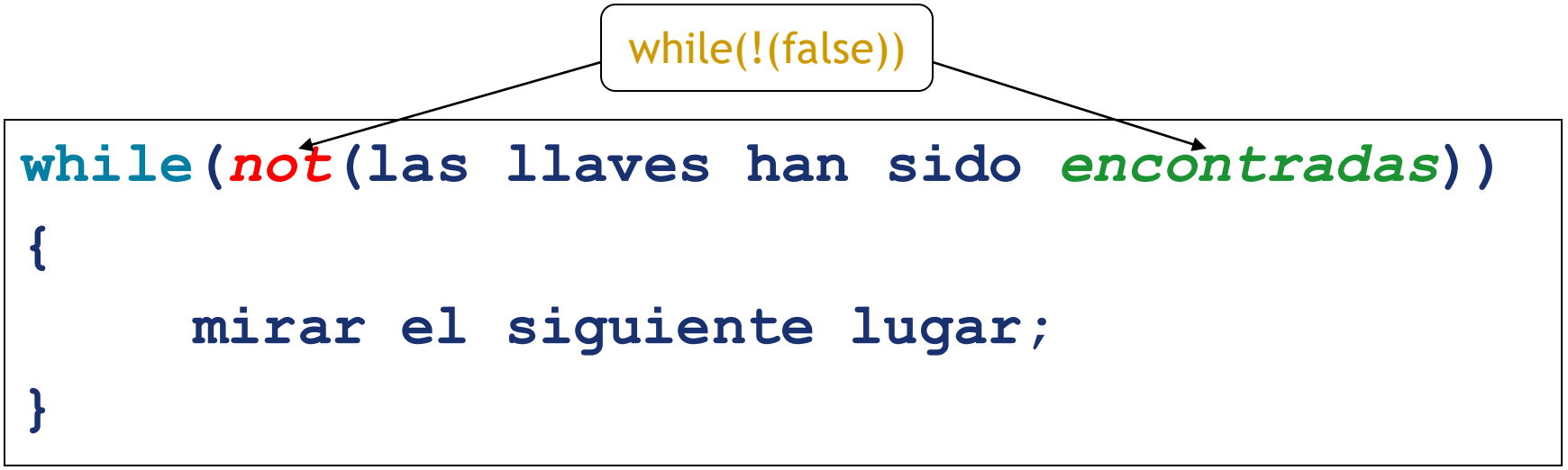
```
for(inicialización;condición de continuación; acción posterior) {
    sentencias a repetir en cada iteración
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Buscando las Llaves (1 de 2)





Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Buscando las Llaves (2 de 2)

```
boolean búsqueda = true;

while(búsqueda)
{
     if(están en el siguiente lugar)
     {
       búsqueda = false;
     }
}
```

¿Supón que no las encontramos?

Bucle infinito

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo de *While*

```
public void listAllFiles()
{
     int index = 0;
     while(index < files.size()) {
          String filename = files.get(index);
          System.out.println(filename);
          index++;
     }
}
```

Incrementa ***index*** en 1

Mientras el valor de *index* es menos que el tamaño de la colección, recupera y muestra el siguiente nombre de fichero y después incrementa *index*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Elementos del While

- Hemos declarado una variable **index**.
- La condición debe declararse correctamente.
- Tenemos que recuperar **cada** elemento.
- La variable index debe ser incrementada de forma explícita.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Equivalencia entre *While* y *For*

```
public void listAllFiles()
{
     int index = 0;
     while(index < files.size()) {
          String filename = files.get(index);
          System.out.println(filename);
          index++;
     }
}
```

Nota: variable index todavía existe ahí …

```
public void listAllFiles()
{
     for(int index = 0;index < files.size(); index++){
          String filename = files.get(index);
          System.out.println(filename);
     }
}
```

Nota: variable index ya no existe ahí …

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Equivalencia entre *While* y *For-Each*

```
public void listAllFiles()
{
     int index = 0;
     while(index < files.size()) {
          String filename = files.get(index);
          System.out.println(filename);
          index++;
     }
}
```

```
public void listAllFiles()
{
     for(String filename : files) {
          System.out.println(filename);
     }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

While vs *For-Each*

- while: **Accede solo ALGUNOS items elementos e incluye un registro del índice de la posición**
  - No tenemos que procesar la colección al completo.
  - Se puede usar no solo con colecciones.
  - **cuidado**: podría producer un ***bucle infinito***.

- for-each: **Accede TODOS los elementos sin cambiar la colección**
  - más fácil de escribir.
  - más seguro: está garantizado que termina.
  - maneja el acceso y recuperación por ti.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Revisión de *While*

- PROS
  - **puede parar en cualquier momento durante el bucle**
  - iteración indefinida de ALGUNOS ítems usando la condición del bucle
  - **puede cambiar la colección durante el bucle**
  - uso de variable explícita de índice dentro y fuera del bucle

- CONS
  - más esfuerzo de codificación
  - requiere declaración de variable índice
  - requiere incremento manual de variable índice
  - requiere determinar correctamente condición de fin
  - debe recuperar el elemento usando el índice
  - **NO garantiza parada con posible bucle infinito**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Búsqueda

- Una actividad recurrente fundamental.
- Aplicable más allá de las colecciones.
- Necesariamente indefinida.
- Debemos codificar tanto para el caso de éxito (para al encontrarlo) como de fallo (después de buscar todo) usando una búsqueda exhaustiva.
  - **Ambos** casos deben hacer que la condición del bucle pase a ser **falsa**, para detener la iteración.
- Incluso debe funcionar si la colección está vacía.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Finalizando una Búsqueda

- ¿Cómo finalizamos una búsqueda?
- **O** no hay más elementos a comprobar:

```
index >= files.size()
```

- **O** el elemento se ha encontrado:

```
found == true
found
! searching
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Continuando una Búsqueda

- Necesitamos establecer la condición de **continuación**:
- Así la condición del bucle será la opuesta de la de finalización:

```
index < files.size() && ! found
index < files.size() && searching
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Condición de Búsqueda

cuando invertir todo

>= pasa a ser <

|| pasa a ser &&

true pasa a ser !true

FINALIZA la búsqueda ***cuando***:

- No hay más ítems o Ítem es encontrado

```
index >= files.size() || found
```

CONTINÚA la búsqueda ***mientras***:

- Hay todavía más ítems e Ítem *no se encuentra*

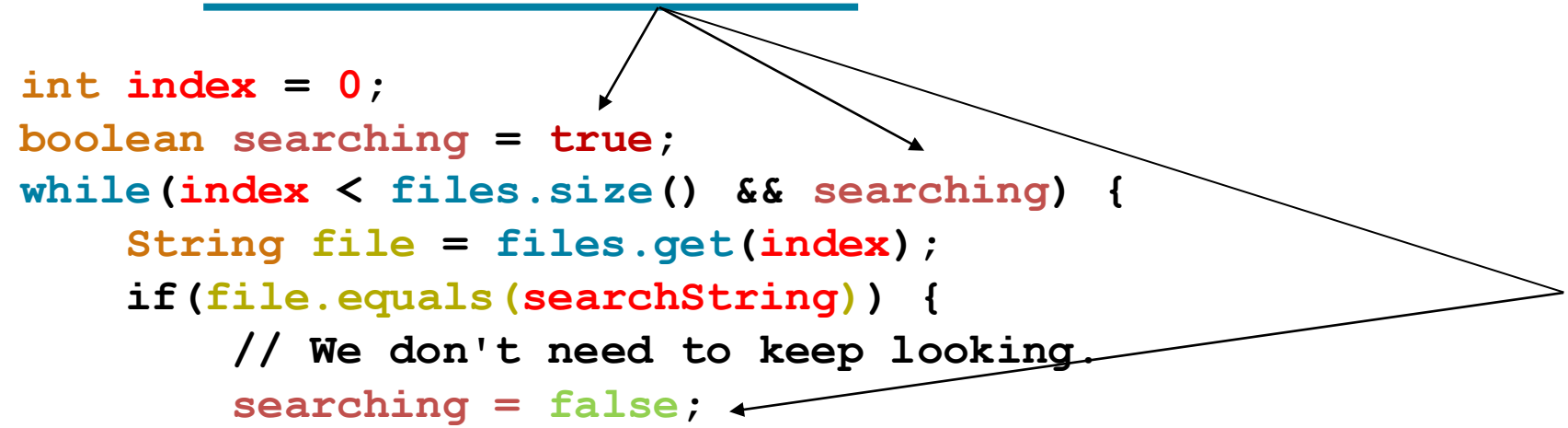
```
index < files.size() && !found
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Buscando en una Colección (1 of 2)

Usando *"todavía buscando"* como condición



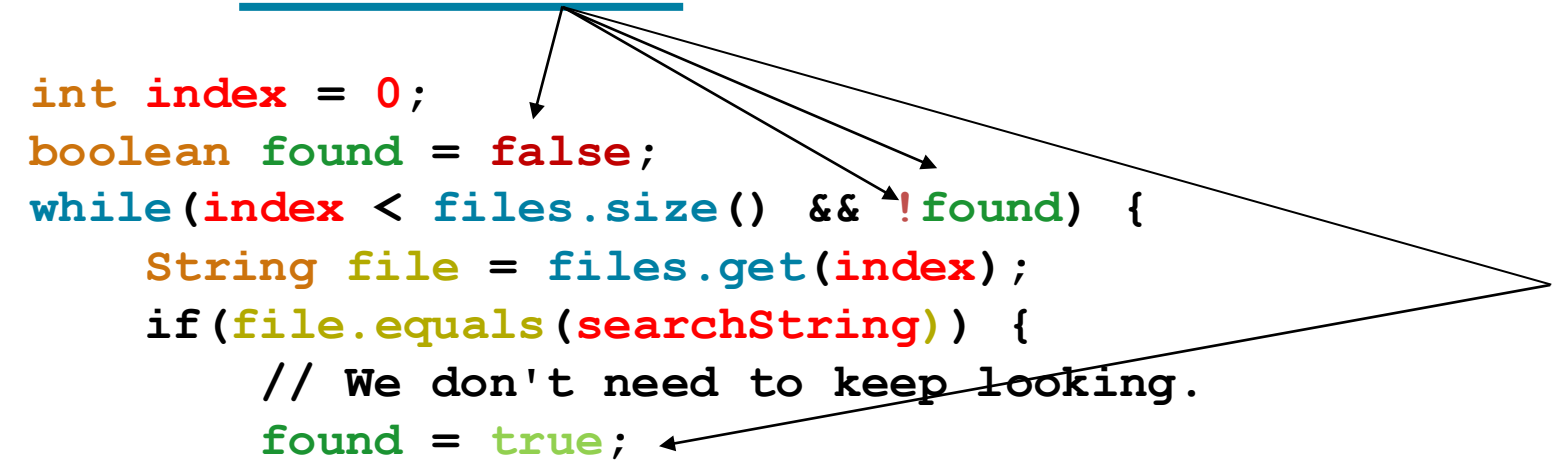
```
int index = 0;
boolean searching = true;
while(index < files.size() && searching) {
    String file = files.get(index);
    if(file.equals(searchString)) {
        // We don't need to keep looking.
        searching = false;
    }
    else {
        index++;
    }
}
// Either we found it at index,
// or we searched the whole collection.
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Buscando en una Colección (2 of 2)

Usando "*encontrado*" como condición



```
int index = 0;
boolean found = false;
while(index < files.size() && !found) {
    String file = files.get(index);
    if(file.equals(searchString)) {
        // We don't need to keep looking.
        found = true;
    }
    else {
        index++;
    }
}
// Either we found it at index,
// or we searched the whole collection.
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Method findFirst

music-organizer-v4

```
public int findFirst(String searchString){
    int index = 0;
    boolean searching = true;

    while(searching && index < files.size())
    {
        String filename = files.get(index);
        if(filename.contains(searchString))
        {                                   // Elemento encontrado
            searching = false;              // Fin búsqueda
        }
        else                                // Elemento no encontrado
        {                                   // Sigue buscando
            index++;                        // Siguiente elemento
        }
    }

    if(searching)                           // Elemento no encontrado
    {
        return -1;                          // Devuelve índice de fuera
    }                                       //  de límite para fallo
    else
    {                                       // Devuelve índice del elemento
        return index;                       //   donde es encontrado
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Iteración Indefinida

- ¿La búsqueda todavía funciona si la colección está vacía?
  - ¡Sí! El cuerpo del bucle no será accedido en ese caso.
- Característica importante del while:
  - El cuerpo puede ser ejecutado **cero o más** veces.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Tipo `Iterator`

- Última variación para iterar sobre una colección
- Se usa un bucle while en combinación con un objeto Iterator
  - NO con una variable índice
- Toma ventaja de la abstracción con el uso de clases de biblioteca (como el *for-each*)
- import *java.util.It*erator;
- clase *Iterator* vs. método *iterator( )*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

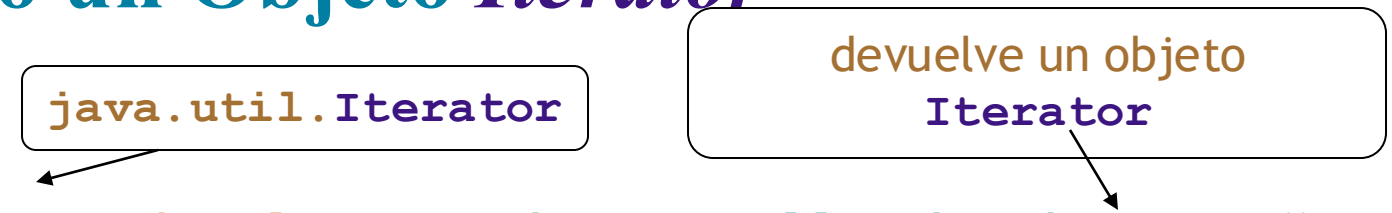
`Iterator` e `iterator()`

- Las colecciones tienen un método `iterator()`.
- Este método devuelve un objeto `Iterator`.
- `Iterator <E>` tiene métodos:
  - `boolean hasNext()`
  - `E next()`
  - `void remove()`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Usando un Objeto *Iterator*




```
Iterator<TipoElemento> it = myCollection.iterator();
while(it.hasNext()) {
     se invoca it.next() para recuperar el siguiente objeto
     se hace algo con ese objeto
}
```

- Declara la variable *it* como tipo *Iterator* de *TipoElemento*
- Usa el método *iterator()* de colección (p.e. ArrayList) y asigna el objeto devuelto *Iterator* a la variable *it*
- el objeto *it* "indexa" el primer elemento de la colección
- *it.hasNext()* comprueba si hay un objeto en el índice
- *it.next()* recuperará el objeto actual y avanzará el índice

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

- Para ver ejemplo más detallado con *Iterator* vamos a usar una versión más avanzada del proyecto music-organizer
- Nuestra colección de objetos String era un poco limitada.
  - private ArrayList<**String**> tracks;
- No separa identificación de artista, título, etc.
- Vamos a usar clase **Track** con campos separados:
  - private String artist;
  - private String title;
  - private String filename;
- Ahora la colección pasa a ser de objetos **Track**
  - private ArrayList<**Track**> tracks;

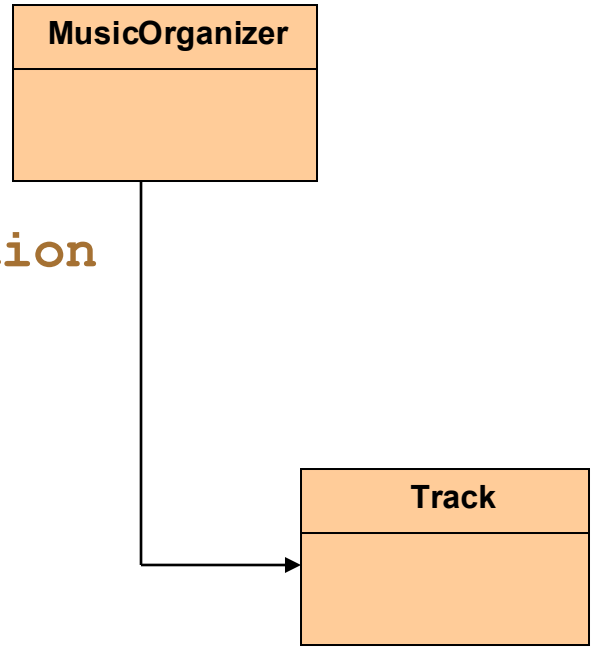
Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

```
public class MusicOrganizer    {
    // ArrayList of Track objects
    private ArrayList<Track> tracks;
        :
        :
}


// non-String Track class definition
public class Track        {
    private String artist;
    private String title;
    private String filename;

        :
        :
}
```



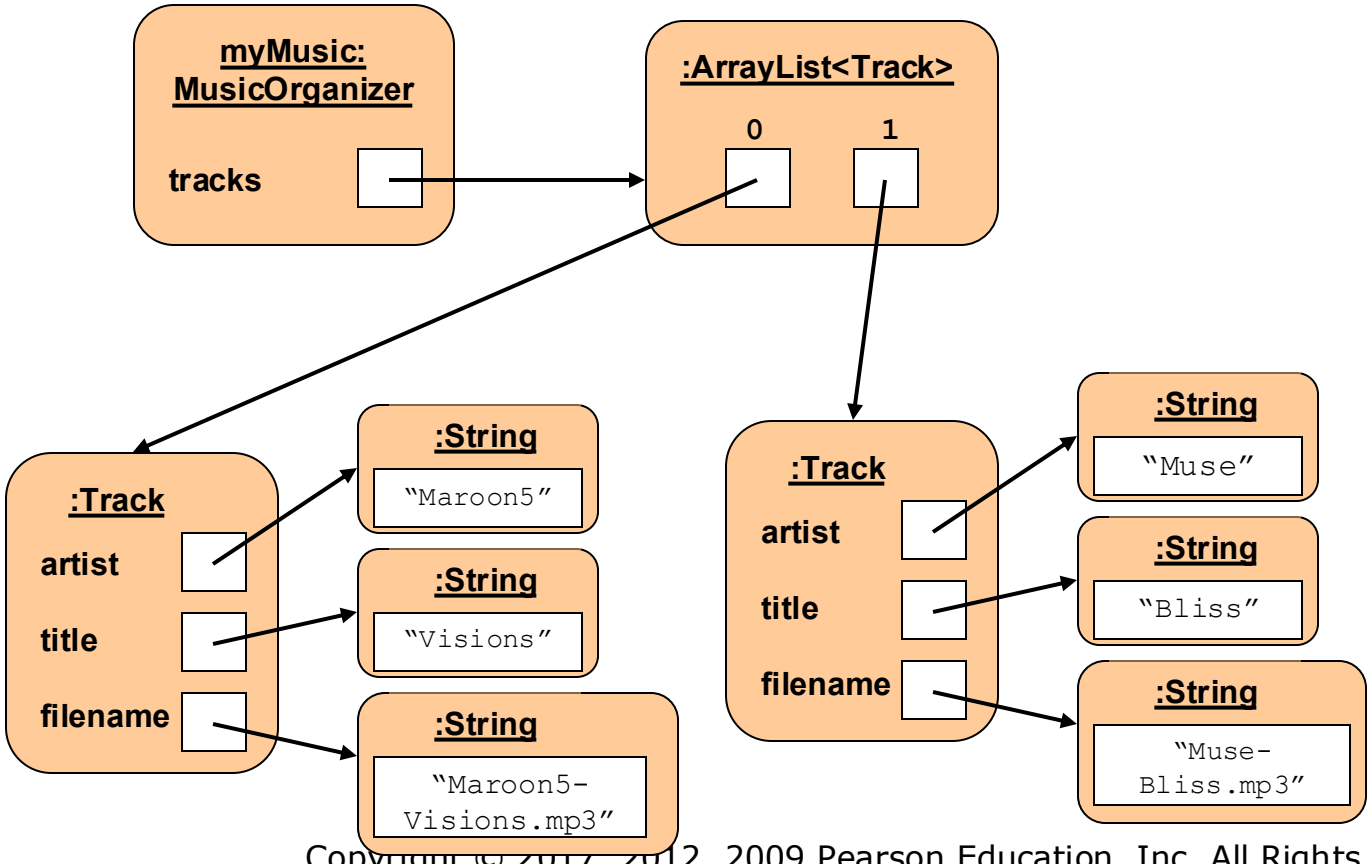
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

Ejemplo de un posible uso de music-organizer-v5 :

- 1 instancia de objeto de la clase *MusicOrganizer myMusic*
- 2 instancias de *Track* en el campo tipo ArrayList de *tracks* de *myMusic*
  - *new Track*(“Maroon5”, “Visions”, “Maroon5-Visions.mp3”)
  - *new Track*(“Muse”, “Bliss”, “Muse-Bliss.mp3”)



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

`java.util.Iterator`

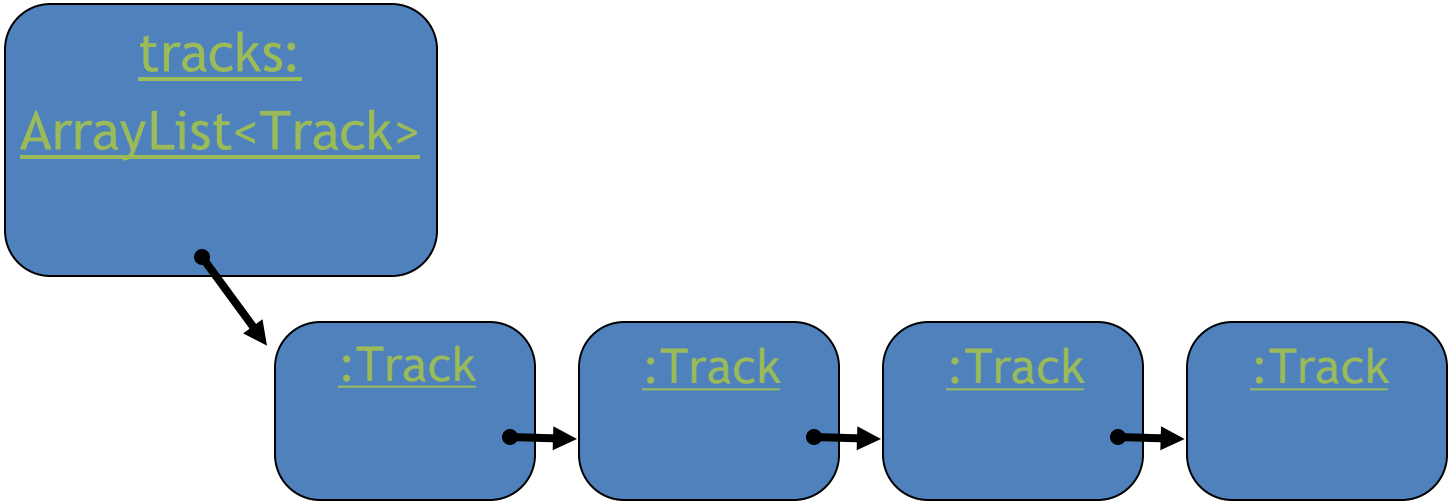
devuelve un objeto *Iterator*

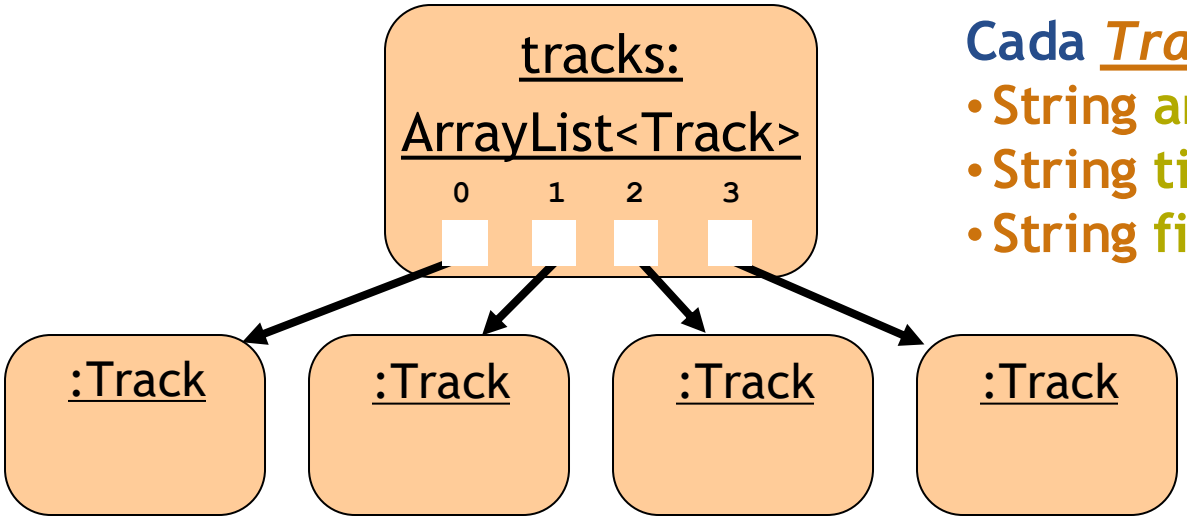
```
public void listAllTracks()
{
    Iterator<Track> it = tracks.iterator();
    while(it.hasNext()) {
        Track tk = it.next();
        System.out.println(tk.getDetails());
    }
```

- Muestra TODAS las pistas en la colección (como while y for-each)
- Todavía usa *while* … PERO no necesita una variable índice
- *Iterator* mantiene la pista de la posición actual, si hay más elementos (*hasNext*) y cuál devolver (*next*)
- *Iterator.next* devuelve el siguiente elemento Y se mueve a la posición siguiente a ese elemento (NO puede retroceder)

Ejemplo Mecánica Objeto *Iterator* con Track

```
private ArrayList<Track> tracks;
```





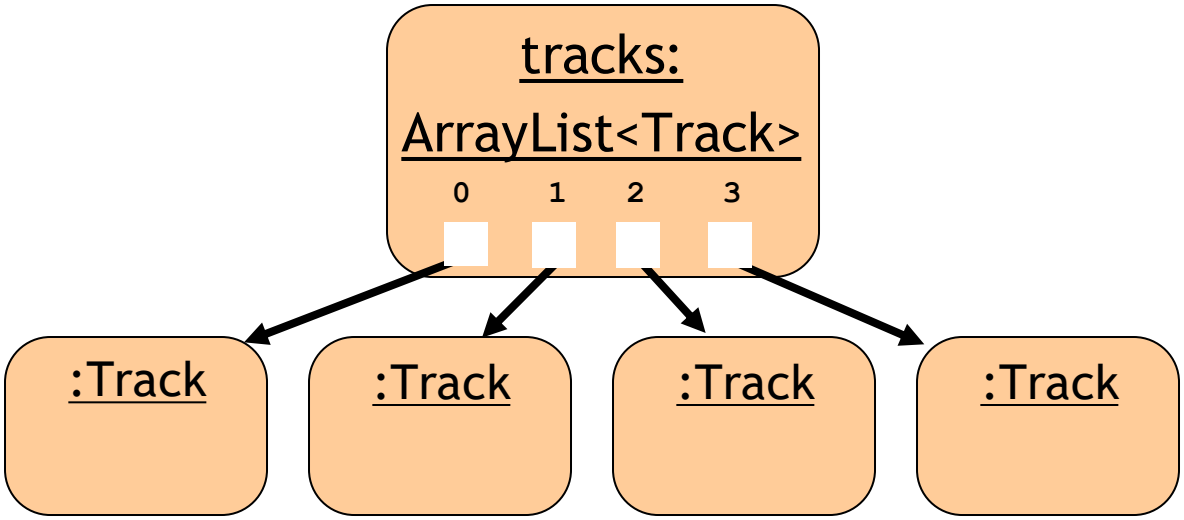
Cada *Track* tiene:

- String artist
- String title
- String filename

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track



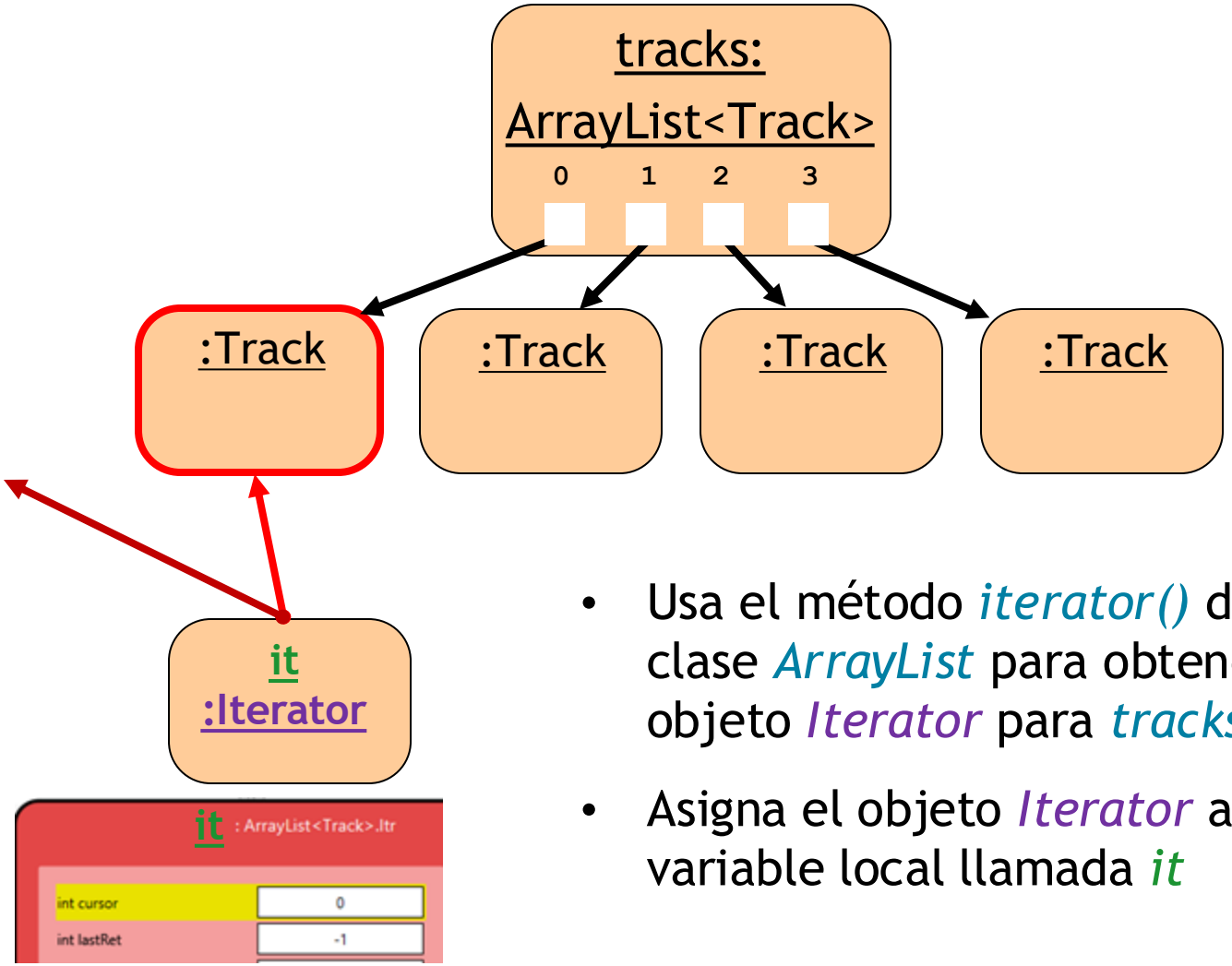

```
public void listAllFiles()
{
    Iterator<Track> it = tracks.iterator();
    while(it.hasNext()) {
        Track tk = it.next();
        System.out.println(tk.getDetails());
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

```
Iterator<Track> it = tracks.iterator();
```



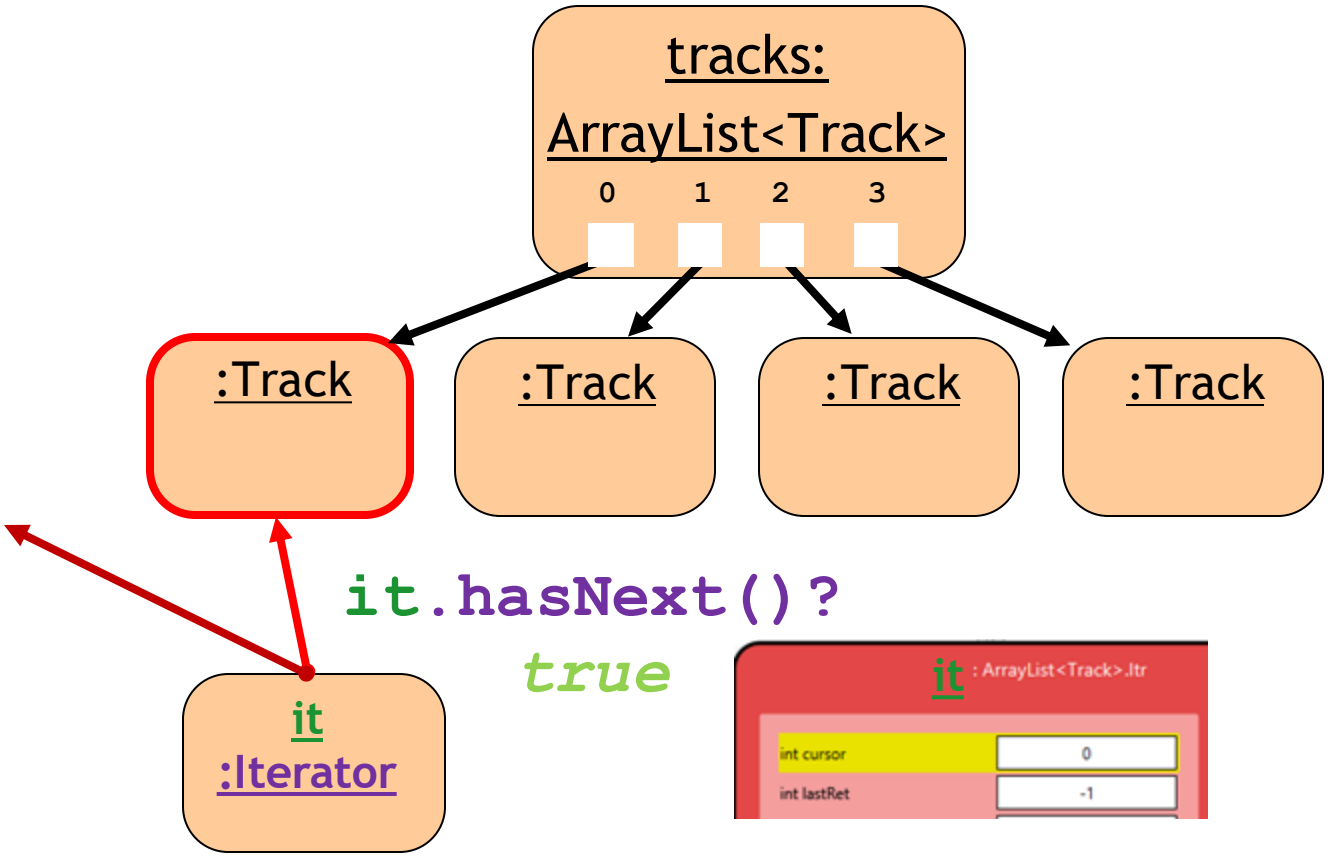
- Usa el método *iterator()* de la clase *ArrayList* para obtener un objeto *Iterator* para *tracks*
- Asigna el objeto *Iterator* a la variable local llamada *it*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

```
while(it.hasNext())
```

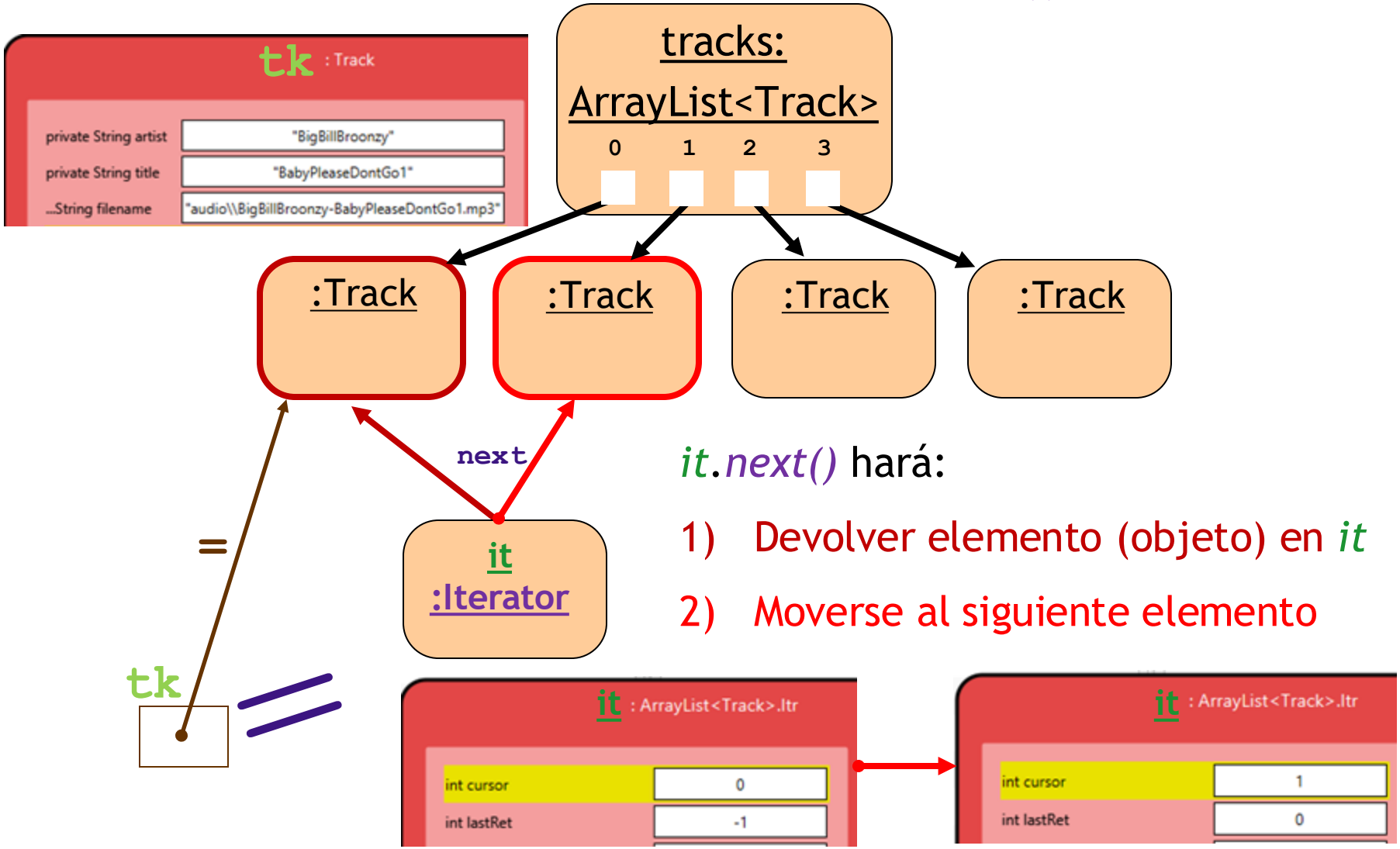


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

```
Track tk = it.next();
```



it.next() hará:

1) Devolver elemento (objeto) en *it*
2) Moverse al siguiente elemento

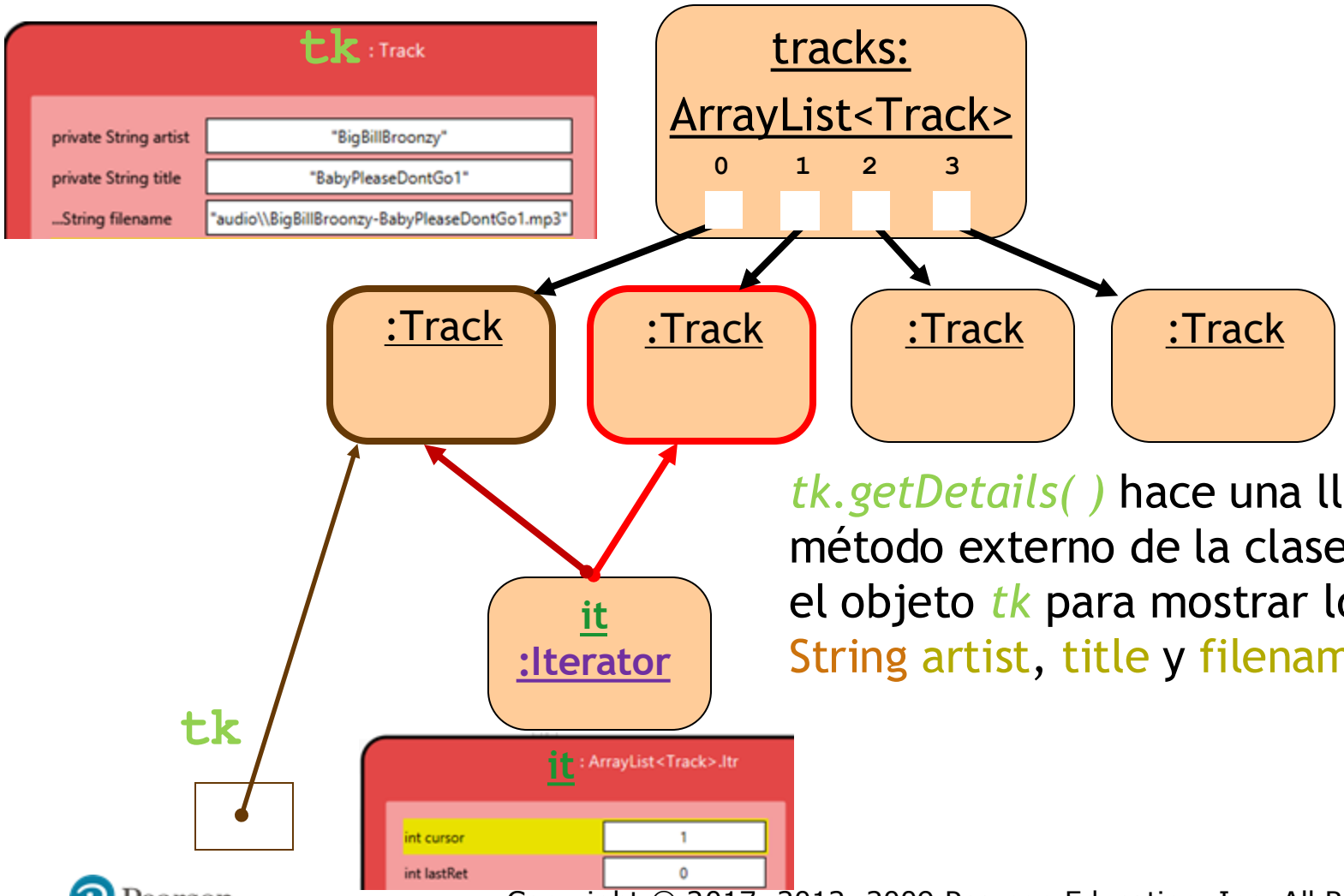
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

```
System.out.println(tk.getDetails());
```

```
BigBillBroonzy: BabyPleaseDontGo1  (file: audio\BigBillBroonzy-BabyPleaseDontGo1.mp3)
```



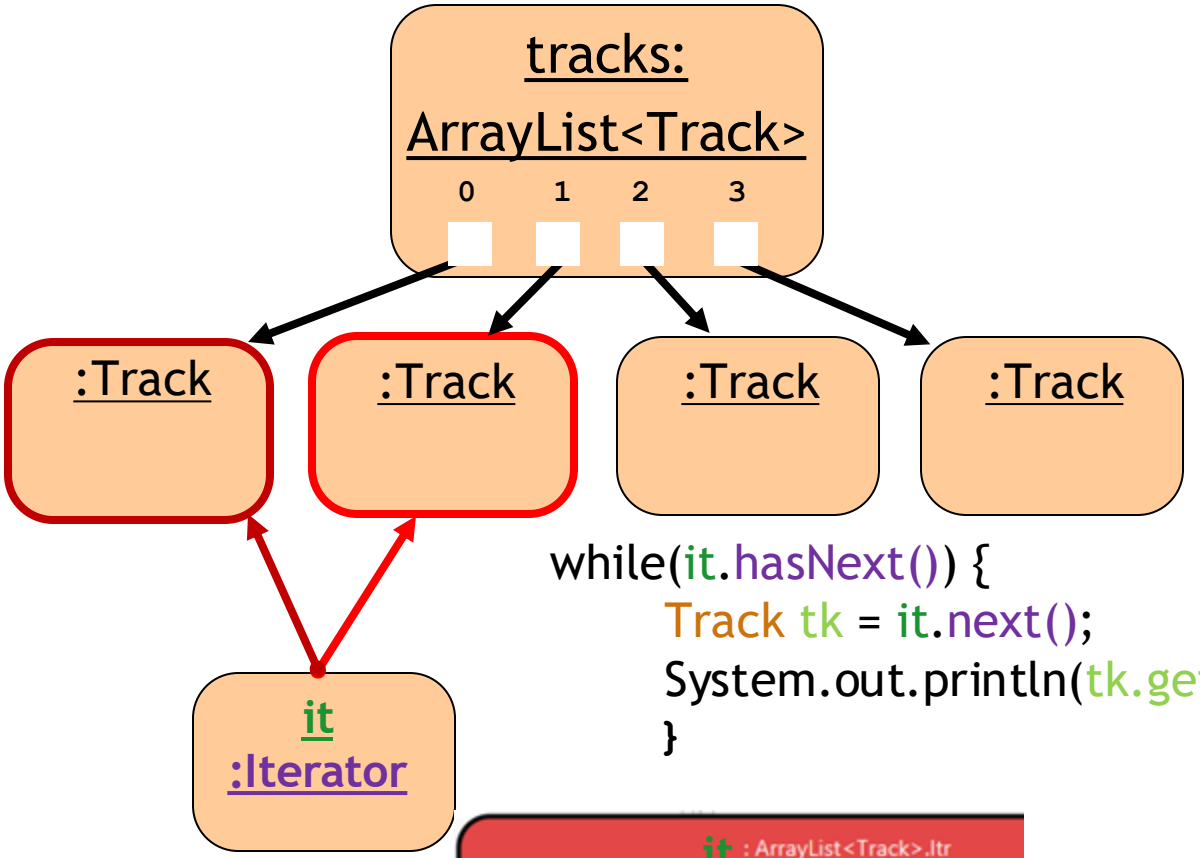
tk.getDetails( ) hace una llamada al método externo de la clase *Track* con el objeto *tk* para mostrar los campos String artist, title y filename.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

Salida de la 1ª iteración del cuerpo del while



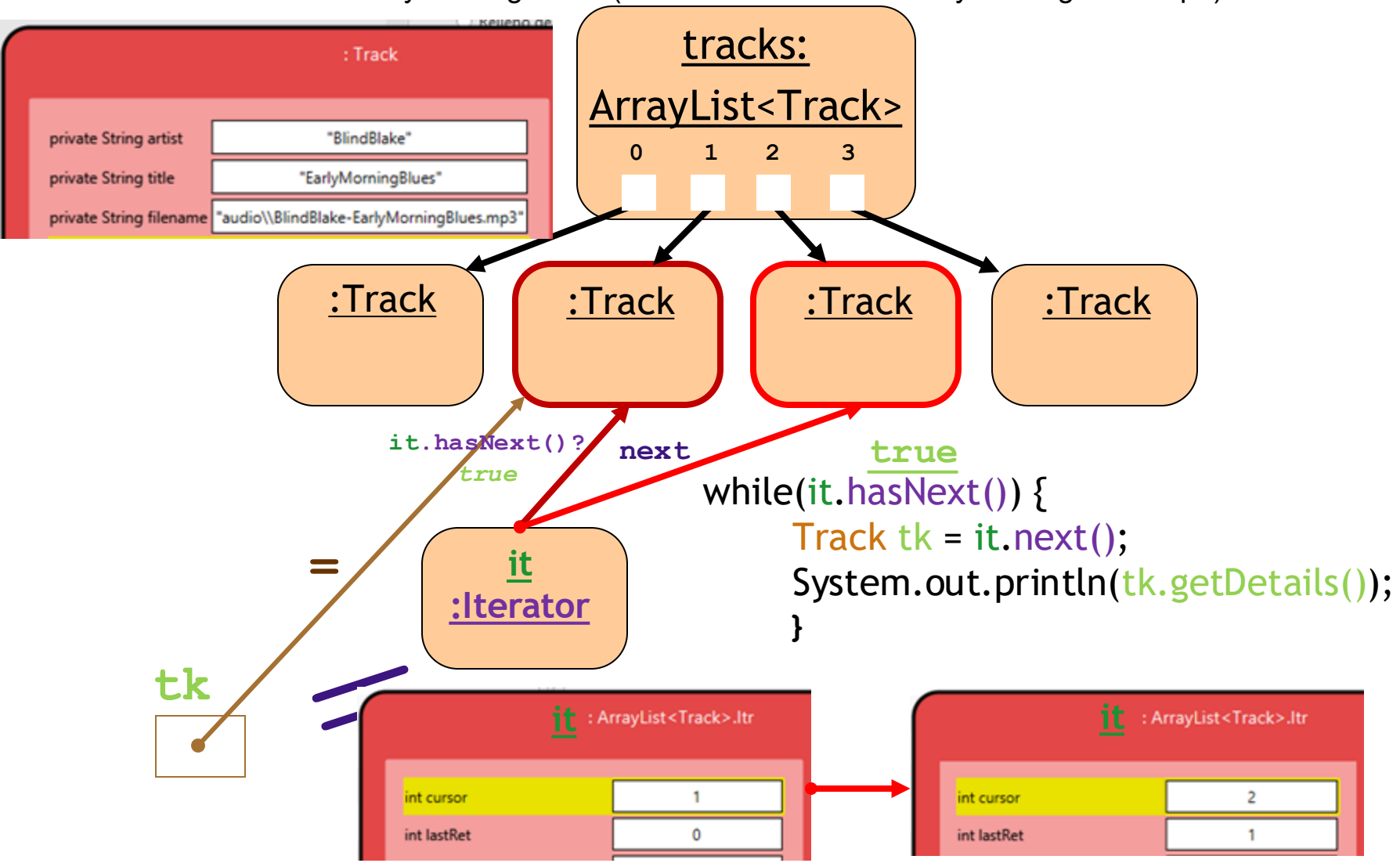
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

2ª iteración

BlindBlake: EarlyMorningBlues (file: audio\BlindBlake-EarlyMorningBlues.mp3)

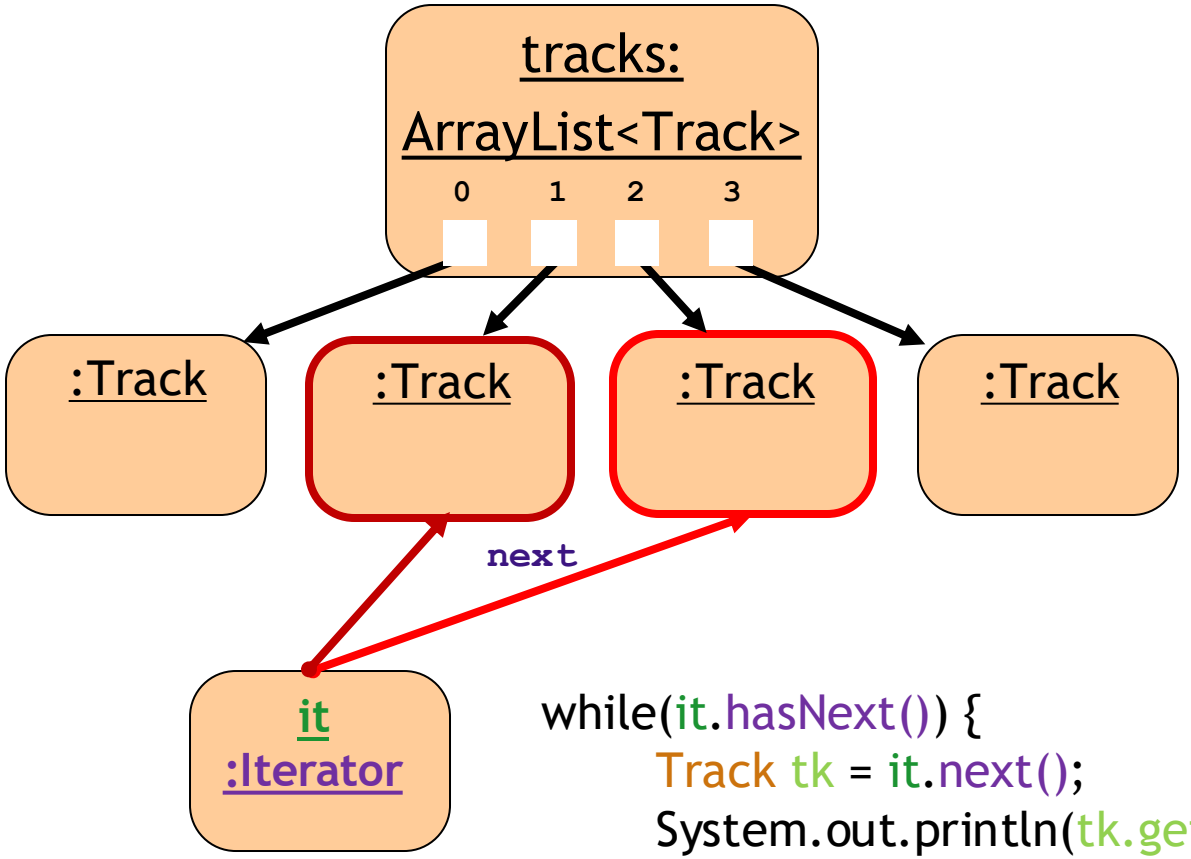


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

Salida 2ª iteración




```
while(it.hasNext()) {
        Track tk = it.next();
        System.out.println(tk.getDetails());
```

tk



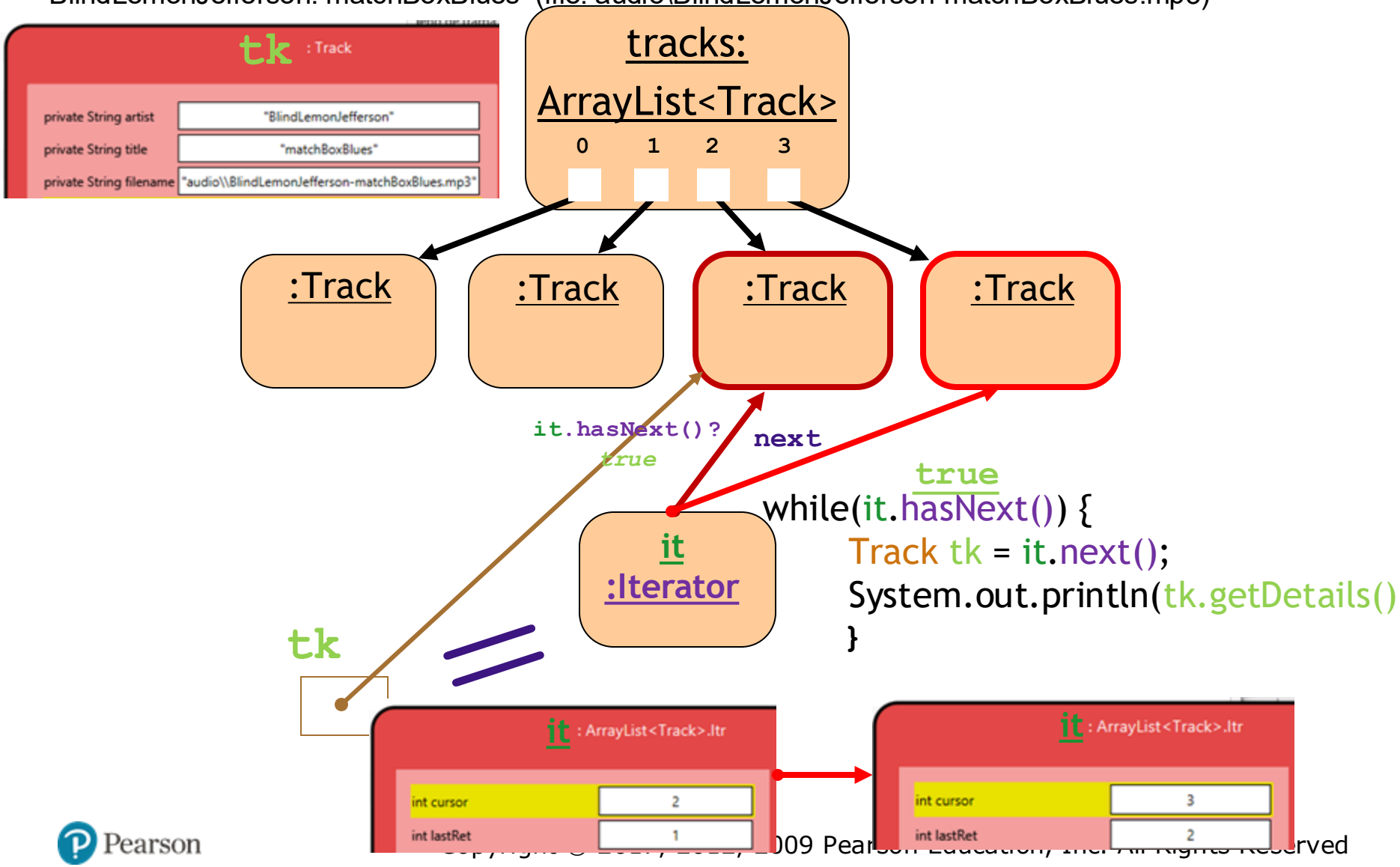
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

3ª iteración

BlindLemonJefferson: matchBoxBlues (file: audio\BlindLemonJefferson-matchBoxBlues.mp3)

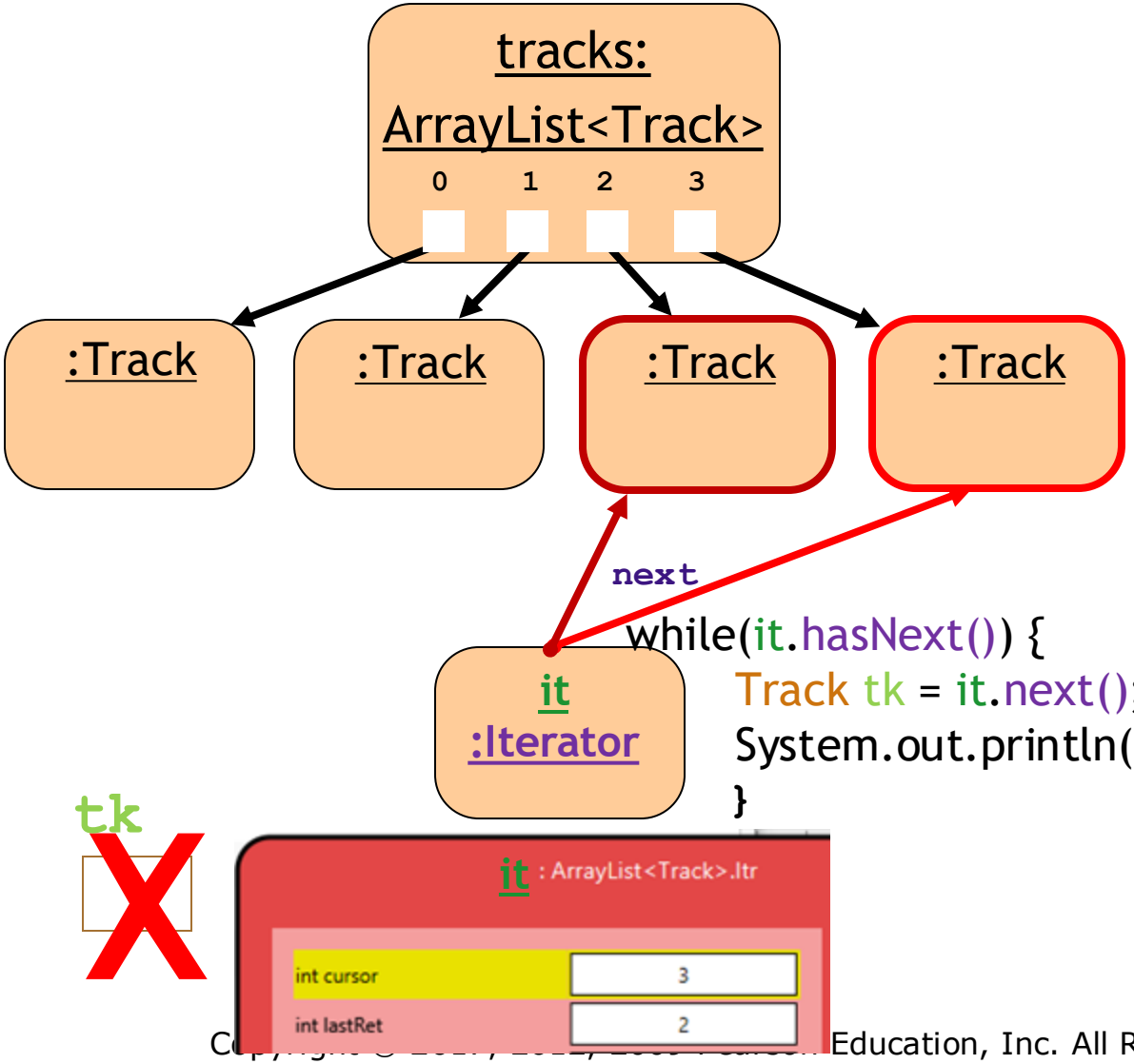


Pearson

009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

Salida 3ª iteración

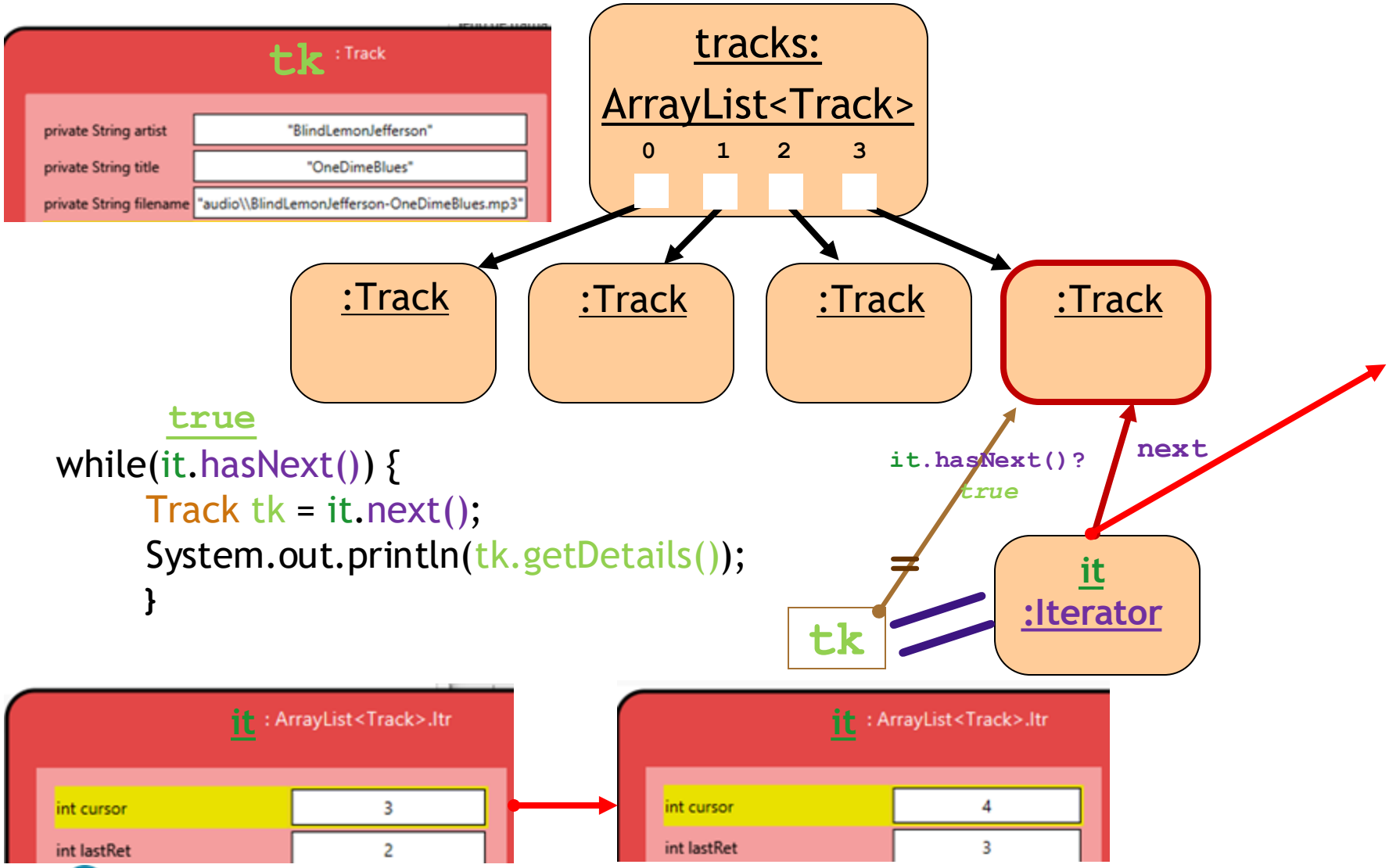


Pearson
Copyright © 2014, 2012, 2010 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

4ª iteración

BlindLemonJefferson: OneDimeBlues (file: audio\BlindLemonJefferson-OneDimeBlues.mp3)

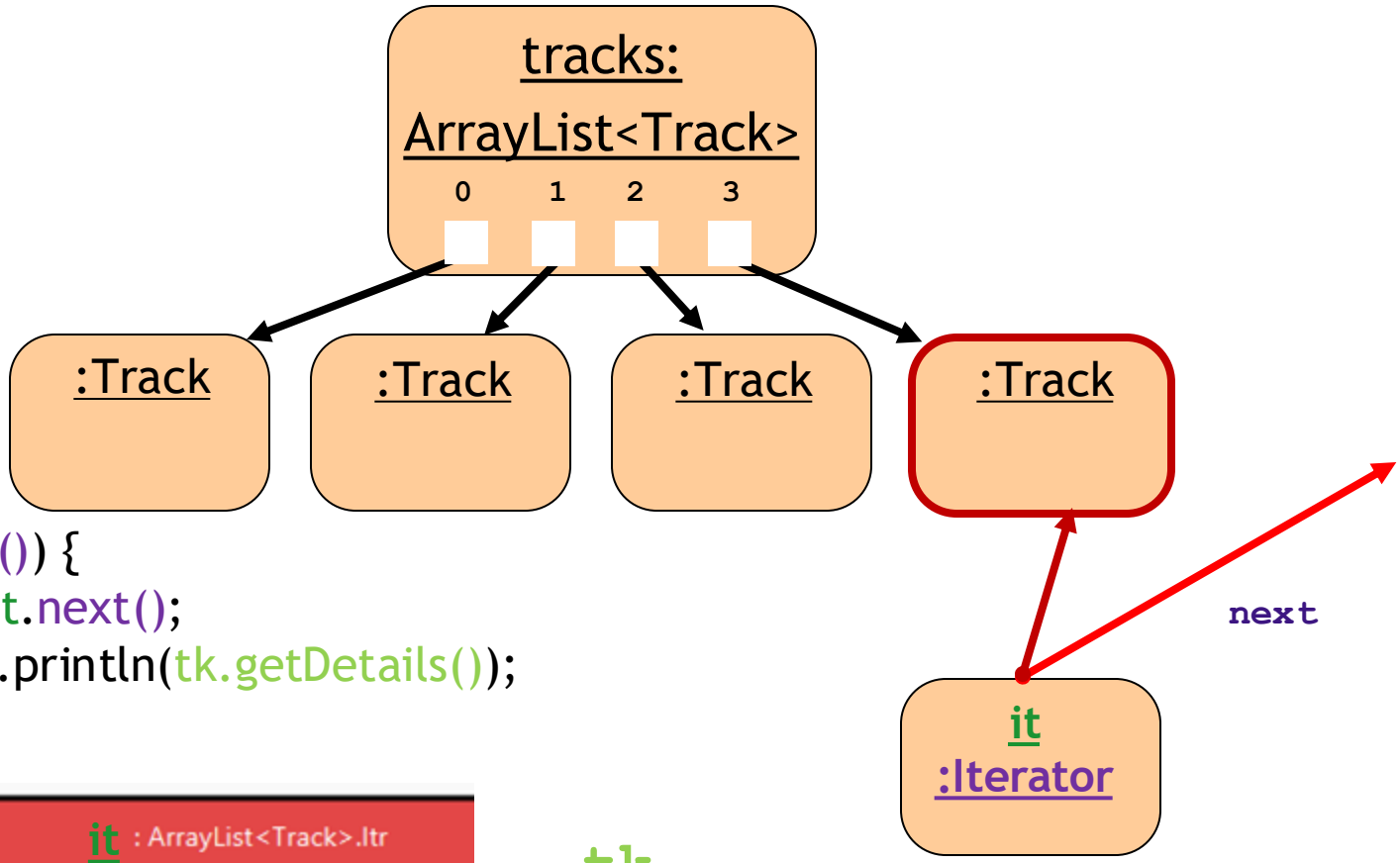


Pearson

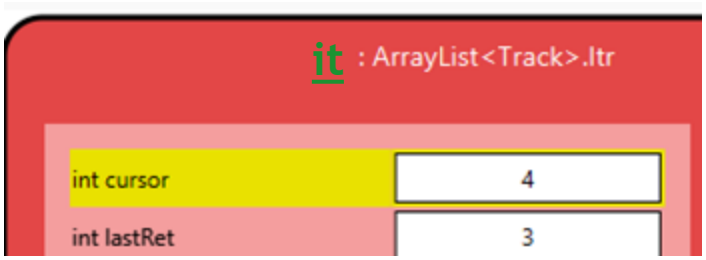
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

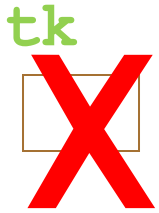
Ejemplo Mecánica Objeto *Iterator* con Track

Salida 4ª iteración




```
while(it.hasNext()) {
        Track tk = it.next();
        System.out.println(tk.getDetails());
        }
```



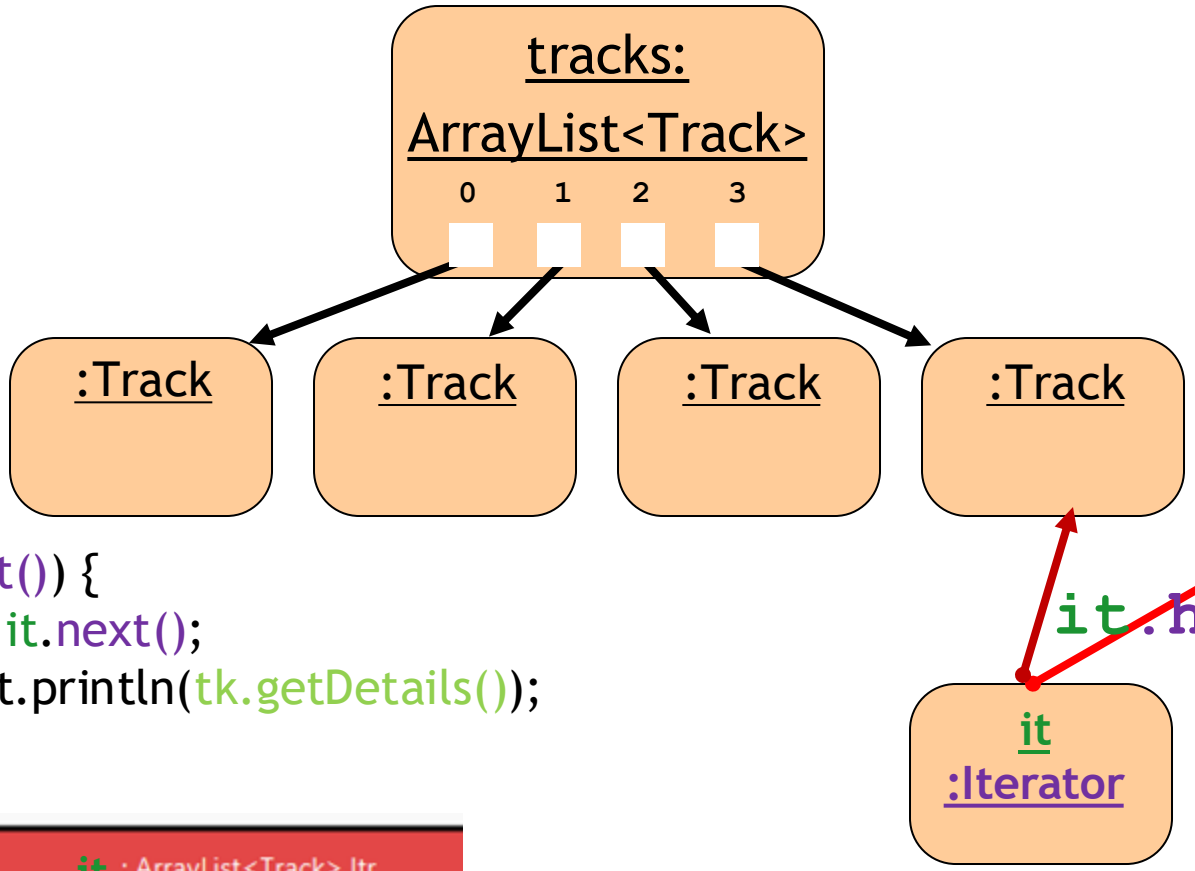


Pearson

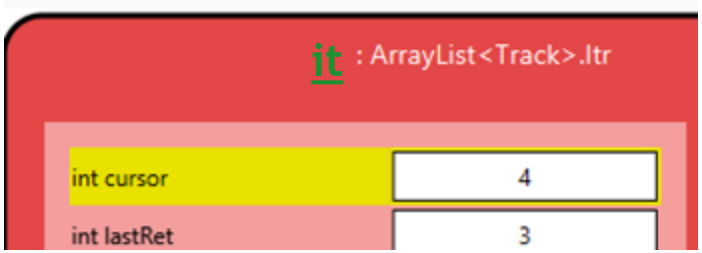
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejemplo Mecánica Objeto *Iterator* con Track

5ª iteración




```
        false
while(it.hasNext()) {
      Track tk = it.next();
      System.out.println(tk.getDetails());
      }
```



NO hay más elementos,
por tanto el bucle while PARA!!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Index Versus *Iterator*

- Formas de iterar sobre una colección
  - bucle for-each *(iteración definida)*.
    - Procesa copia de cada elemento de la colección.
  - bucle while *(iteración indefinida)*.
    - Usado cuando podríamos querer parar a medio camino.
    - También usable para bucles que no involucran colecciones.
  - objeto `Iterator` *(iteración indefinida)*.
    - Usado cuando podríamos querer parar a medio camino.
    - Usado a menudo con colecciones donde el acceso indexado no es eficiente o no es posible.
    - Disponible para todas las colecciones de la biblioteca Java
    - **Usado para eliminar elementos de una colección.**
- Iteration es un importante **patrón de diseño**.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminando elementos de una Colección

```
para cada track en la colección tracks
{
     si track.getArtist( ) ya no es de nuestro gusto:
       tracks.remove(track)
}
```

- Imposible con un bucle *for-each*

```
public void removeArtist(String artistToRemove)
{
    for(Track track : tracks) {
        if(track.getArtist().contains(artistToRemove)){
        tracks.remove(track);}
```

  - De inicio borra el objeto **`track`** y parece que todo va bien
  - Pero al continuar la iteración causa **¡¡ConcurrentModificationException!!**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminando elementos de una Colección

```
para cada track en la colección tracks {
    si track.getArtist( ) ya no es de nuestro gusto:
      tracks.remove(track)
}
```

- Bucle *while* posible, pero NO recomendable

```
int index = 0;
while(index < tracks.size()) {
    Track t = tracks.get(index);
    String artist = t.getArtist();
    if(artist.equals(artistToRemove)) {
        tracks.remove(index);
    }
    index++;
}
```

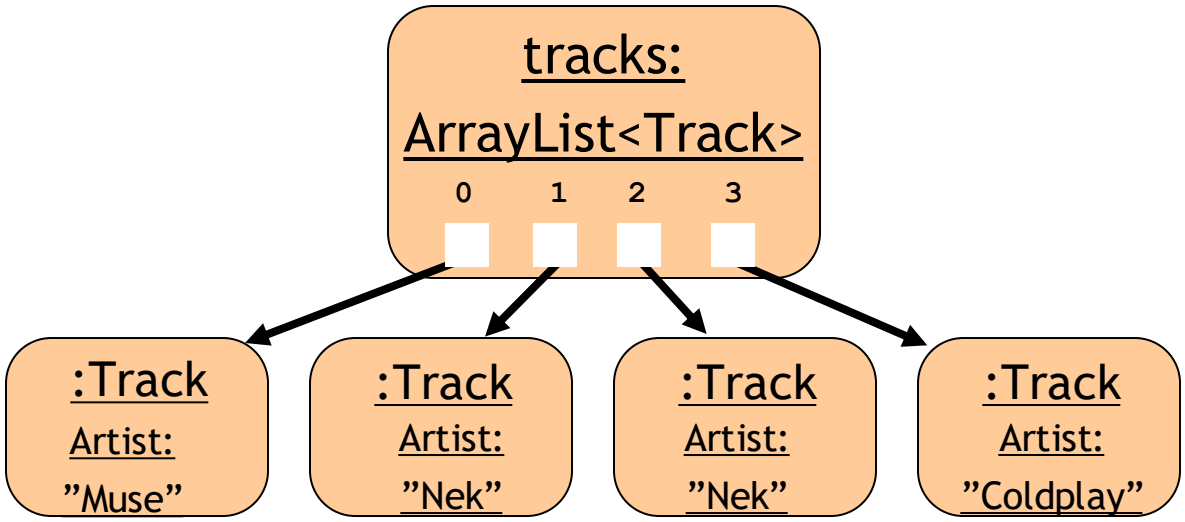
¿Puedes suponer lo que está mal?

PISTA: ¿Qué ocurre si se elimina el penúltimo elemento? ¿Se comprobaría el último elemento?

¡¡¡EPIC FAIL!!!

¡¡ Solución correcta es usar while con *Iterator* !!

Eliminando elementos de una Colección




```
int index = 0;
while(index < tracks.size()) {
     Track t = tracks.get(index);
     String artist = t.getArtist();
     if(artist.equals(artistToRemove)) {
          tracks.remove(index);
     }
     index++;
}
```

"Nek"

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminando elementos de una Colección

```
Iterator<Track> it = tracks.iterator();
while(it.hasNext()) {
     Track t = it.next();
     String artist = t.getArtist();
     if(artist.equals(artistToRemove)) {
          it.remove();
     }
}
```

Usa el método `remove` de `Iterator`.

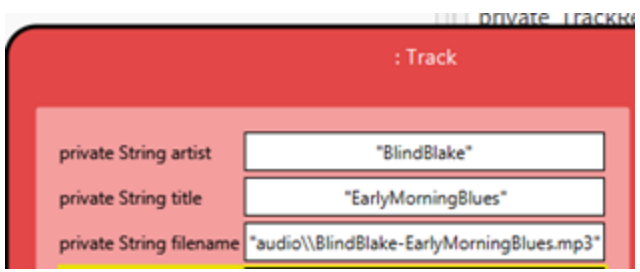
- **NO** usa la variable de colección *tracks* en el cuerpo del bucle
- **Debe usarse** *remove( )* **de** *Iterator* **y NO de** *ArrayList*
- *Iterator* **solo** *elimina* **el último elemento recuperado con** *next*
- *Iterator* **nos oculta los detalles de la eliminación y mantiene la iteración sincronizada**

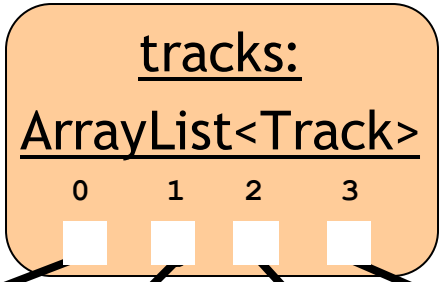
Pearson

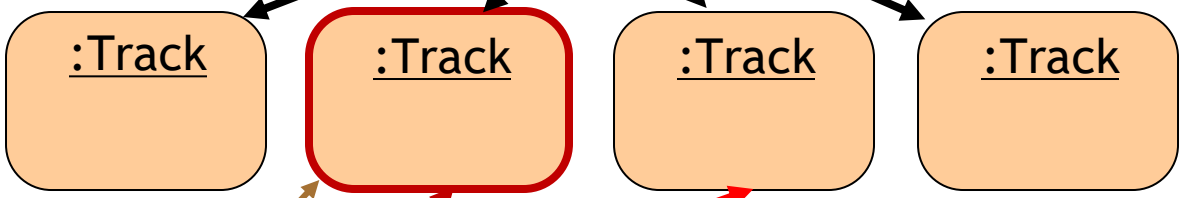
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

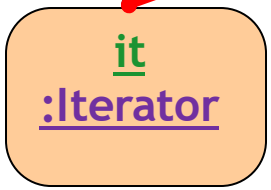
Eliminando elementos de una Colección

2ª iteración

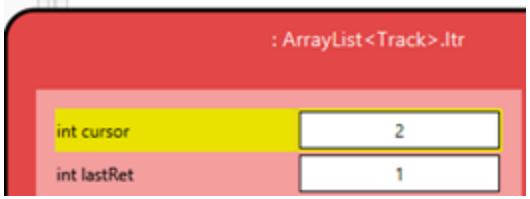












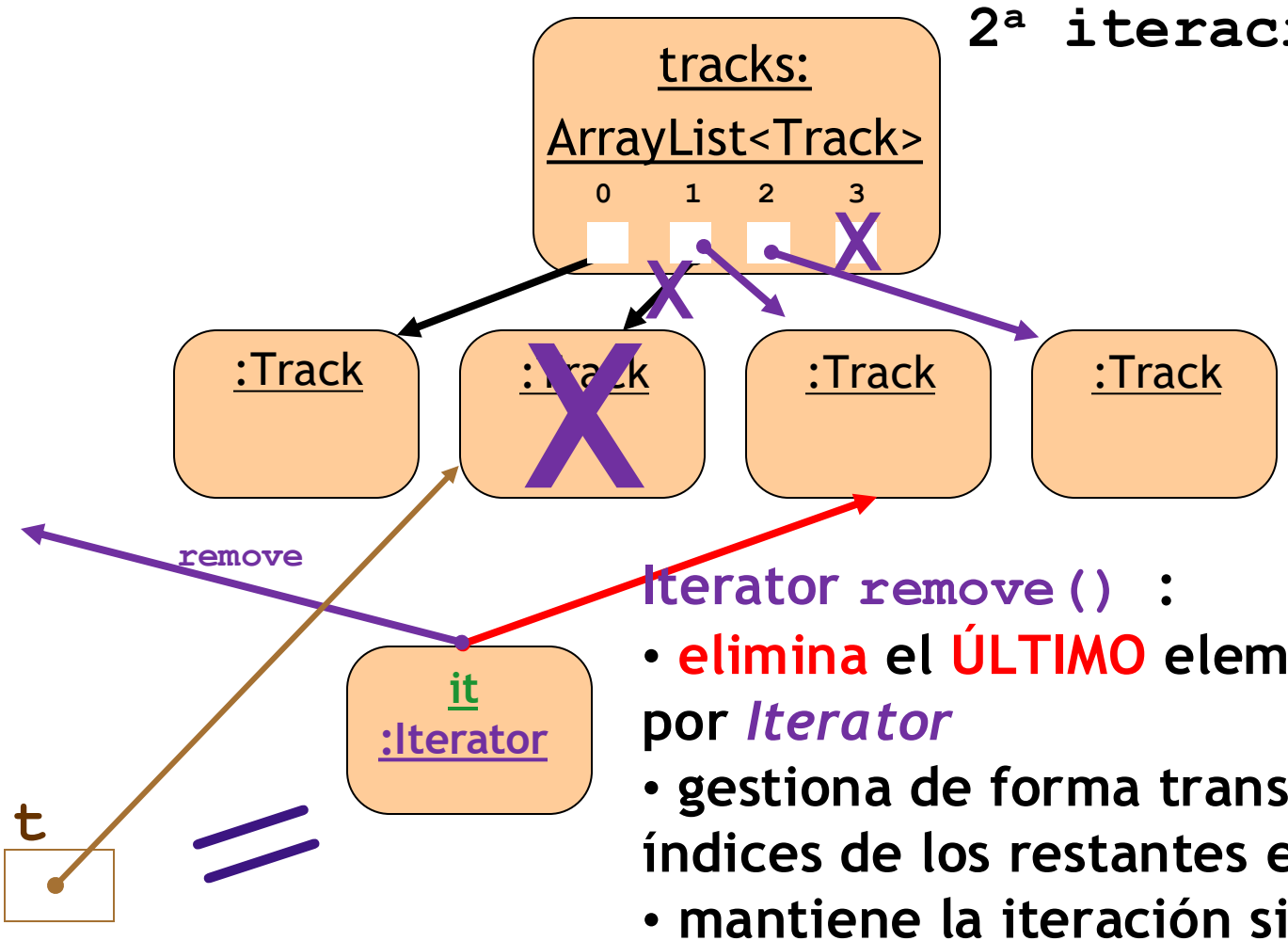

```
while(it.hasNext()) {
     Track t = it.next();
     String artist = t.getArtist();
     if(artist.equals(artistToRemove)) {
          it.remove();
     }
}
```

"BlindBlake"

Pearson

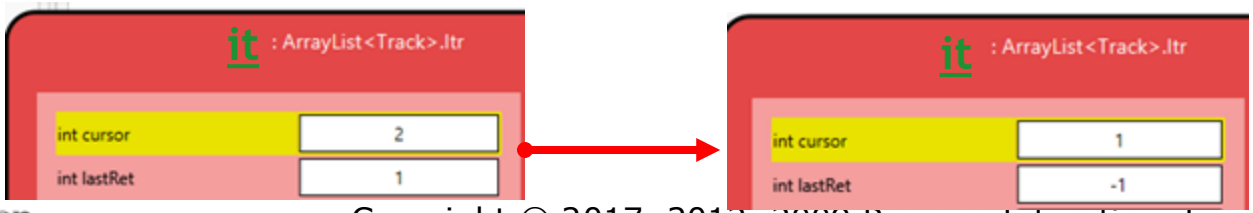
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminando elementos de una Colección



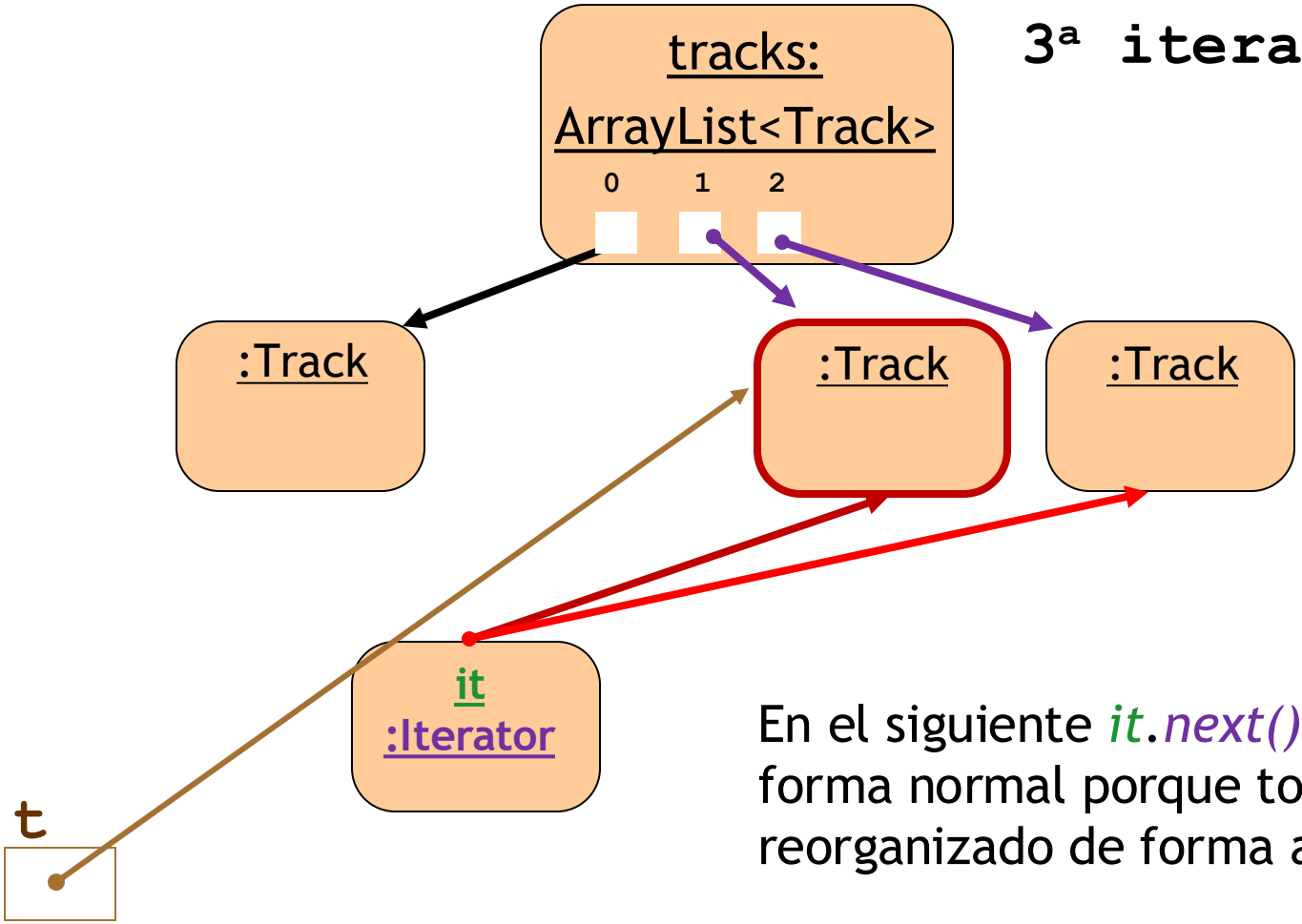
Iterator `remove()` :

- elimina el ÚLTIMO elemento devuelto por *Iterator*
- gestiona de forma transparente los índices de los restantes elementos
- mantiene la iteración sincronizada

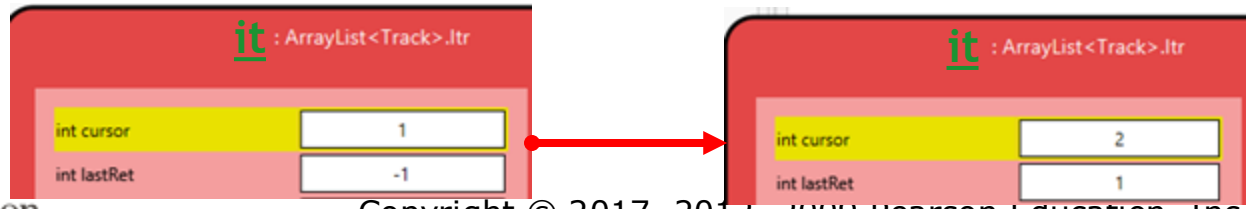


Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Eliminando elementos de una Colección



En el siguiente *it.next()* ya sigue de forma normal porque todo se ha reorganizado de forma automática



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Bucle *For* e *Iterator*

- ¿Podemos usar un bucle *for* con un *Iterator* para acceder CADA element en una colección y ELIMINAR selectivamente elementos? SÍ
  - Se puede utilizar un uso especial del bucle *for*:

```
for(Iterator<Track> it = tracks.iterator(); it.hasNext(); ){
    Track track = it.next();
    if(track.getArtist().equals(artistToRemove))
    {
        it.remove();
    }
}
```

No post-body action required.

- NO hay acción al final de cada iteración en la cabecera porque el incremento está siendo gestionado por *it.next* en el cuerpo del bucle
  (Pero sigue siendo necesario poner ";")

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

while versus *do-while*

```
Iterator<ElementType> it = myCollection.iterator();
while(it.hasNext()) {
     invocar it.next() para recuperar el siguiente objeto
     hacer algo con ese objeto
        posibilidad de usar it.remove()
}
```

¿Cómo es de diferente un bucle *do-while*?

```
si la colección tiene al menos 1 elemento
{
  Iterator<ElementType> it = myCollection.iterator();
  do {
     invocar it.next() para recuperar el siguiente objeto
     hacer algo con ese objeto
         posibilidad de usar it.remove()
  } while(it.hasNext());
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Qué bucle debería usar?

- *for* :
  - iteración definida con inicio y fin conocido
  - incremento puede ser flexible (> 1)
  - es necesario un contador (índice).
- *for-each*:
  - itera sobre TODOS los elementos en la colección
  - no se pueden añadir, eliminar o modificar elementos
  - no se necesita contador (índice).
- *while*:
  - indefinido con número desconocido de iteraciones
  - el fin del bucle se determina con una(s) condicione(s)
- *sobre estructuras que no sean colecciones (Collections)*:
  - hay que usar *for* o *while*
- *Eliminar elementos*:
  - (si se recorren TODOS los elementos): *for con Iterator*
  - (si se para durante el bucle): *while* con *Iterator*

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Ordenando colecciones de elementos (1 de 8)

- Comparator. Ejemplo: proyecto **StudentComparator**

```
class Student {
    private int rollno;   //roll number
    private String name;  //name of the student
    private int age;      //age of the student
   //…
  }
```

¿Podemos ordenar los elementos de studentList?

```
class StudentOrderCheck{
   public static void main(String args[]){
      //Creating a list of students
      List<Student> studentList=new ArrayList<Student>();
      studentList.add(new Student(101,“Ana",23));
      studentList.add(new Student(106,“Pepe",27));
      studentList.add(new Student(105,"Laura",21));
      for(Student st : studentList){
            System.out.println(st.getRollno()+" "+st.getName()+" "+st.getAge());
      }
}
```

Ref: https://www.javatpoint.com/difference-between-comparable-and-comparator

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ordenando colecciones de elementos (2 de 8)

- Comparator. Ejemplo: proyecto **StudentComparator**

```
class AgeComparator implements Comparator<Student>{
    public int compare (Student s1,Student s2){
        if(s1getAge()==s2.getAge())
    return 0;
        else if(s1.getAge()>s2.getAge())
    return 1;
        else
    return -1;
    }
}

class NameComparator  implements Comparator<Student> {

public int compare (Student s1,Student s2){
    return s1.getName().compareTo(s2.getName());
    }
}
```

Interface* para ordenar elementos (java.util)

Método que ordena elementos (s1, s2: objetos a comparar)

La clase String proporciona su propio comparador

¡¡Podemos establecer diferentes criterios de ordenación!!

*= detalles de qué es un Interface se verán en la Unidad sobre "Herencia"

Ordenando colecciones de elementos (3 de 8)

```
class StudentOrderCheck {
  public static void main(String args[]){
    List<Student> studentList=new ArrayList<Student>();
    studentList.add(new Student(101,“Ana",23));
    studentList.add(new Student(106,“Pepe",27));
    studentList.add(new Student(105,"Laura",21));
```

```
    System.out.println("Sorting by Name");
  //Using NameComparator to sort the elements
    Collections.sort(studentList, new NameComparator());
    for(Student st : studentList){
        System.out.println(st.getRollno()+" "+st.getName()+" "+st.getAge());
    }
```

```
    System.out.println(“Sorting in reverse order by Age");
  //Using AgeComparator to sort the elements in reverse order
    Collections.sort(studentList, Collections.reverseOrder(new AgeComparator()));
  //Traversing the list again
     for(Student st : studentList){
         System.out.println(st.getRollno()+" "+st.getName()+" "+st.getAge());
     }
```

Salida:

```
Sorting by Name
101 Ana 23
105 Laura 21
106 Pepe 27
Sorting in reverse order
by Age
106 Pepe 27
101 Ana 23
105 Laura 21
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ordenando colecciones de elementos (4 de 8)

- Comparator. Ejemplo: proyecto **StudentComparatorWithEqualities**

```
class Student {
    private int rollno;   //roll number
    private String name;  //name of the student
    private int age;      //age of the student
   //…
  }
```

¿Podemos añadir un segundo criterio para cuando los valores del primero son iguales?

```
class StudentOrderCheck{
   public static void main(String args[]){
      //Creating a list of students
      List<Student> studentList=new ArrayList<Student>();
      studentList.add(new Student(101,“Ana",23));
      studentList.add(new Student(106,“Pepe",27));
      studentList.add(new Student(105,"Laura",27));
      for(Student st : studentList){
            System.out.println(st.getRollno()+" "+st.getName()+" "+st.getAge());
      }
}
```

Ref: https://www.javatpoint.com/difference-between-comparable-and-comparator

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ordenando colecciones de elementos (5 de 8)

- Comparator. Ejemplo: proyecto **StudentComparatorWithEqualities**

```
class AgeComparator implements Comparator<Student>{
    public int compare (Student s1,Student s2){
        if(s1getAge()==s2.getAge())
    return new RollnoComparator().compare(s1, s2);
        else if(s1.getAge()>s2.getAge())
    return 1;
        else
    return -1;
     }
}
```

En caso de igualdad en la edad, en lugar de devolver 0 y que aplique orden por defecto, ahora desempatamos por el criterio del número de matrícula

```
class RollnoComparator implements Comparator<Student>{
    public int compare (Student s1,Student s2){
        if(s1getRollno () == s2.getRollno ())
    return 0;
        else if(s1.getRollno ()>s2.getRollno ())
    return 1;
        else
    return -1;
    }
}
```

¡¡Podemos establecer diferentes criterios anidados de ordenación para cuando los valores comparados son iguales!!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ordenando colecciones de elementos (6 de 8)

```
class StudentOrderCheck {
  public static void main(String args[]){
    List<Student> studentList=new ArrayList<Student>();
    studentList.add(new Student(101,“Ana",23));
    studentList.add(new Student(106,“Pepe",27));
    studentList.add(new Student(105,"Laura",27));
```

```
    System.out.println("Sorting by Age ");
  //Using AgeComparator to sort the elements
    Collections.sort(studentList, new AgeComparator());
    for(Student st : studentList){
        System.out.println(st.getRollno()+" "+st.getName()+" "+st.getAge());
    }
```

```
    System.out.println(“Sorting in reverse order by Age");
  //Using AgeComparator to sort the elements in reverse order
    Collections.sort(studentList, Collections.reverseOrder(new AgeComparator()));
  //Traversing the list again
     for(Student st : studentList){
         System.out.println(st.getRollno()+" "+st.getName()+" "+st.getAge());
     }
```

Salida:

```
Sorting by Age
101 Ana 23
105 Laura 27
106 Pepe 27
Sorting in reverse order by
Age
106 Pepe 27
105 Laura 27
101 Ana 23
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ordenando colecciones de elementos (7 de 8)

```
/**
 * Java class to demonstrate the use of Java Comparator.
 * Creating a class which implements Comparator Interface
 * (using age field)
 */
import java.util.*;

class AgeComparator implements Comparator <Student>
{

public int compare (Student s1,Student s2)
{
      if(s1.getAge() == s2.getAge())
          return 0;
      else if(s1.getAge() > s2.getAge())
          return 1;
      else return -1;
}

}
```

Interface Comparator

Objetos de la clase que se van a ordenar

Ordenación en orden creciente

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ordenando colecciones de elementos (8 de 8)

Características de **Comparator**

- Pertenece al paquete java.util
- Proporciona múltiples secuencias de ordenación.
  - Es decir, podemos ordenar colecciones en función de varios atributos como p.ej. id, nombre, etc.
- No afecta a la clase original
  - Es decir, la clase ordenada no se modifica.
- Proporciona el método compare() para ordenar elementos.
- Podemos ordenar la lista de elementos de tipo Comparator usando el método Collections.sort(List, Comparator)
  - O en orden inverso con:

```
Collections.sort(List,Collections.reverseOrder(Comparator));
```

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Desordenando colecciones de elementos

```
import java.util.Collections;
ArrayList<String> files = new ArrayList<>();
Collections.shuffle(files);
```

Desordenando una colección:

- Importan la clase *Collections* del paquete *java.util*
- Invocan el método *shuffle* para ordenar de forma aleatoria los elementos existentes
  - Parámetro – pasa la colección entera

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales 1

- Colecciones de tamaño fijo
  - **`Arrays`**
- Colecciones
  - **`ArrayList`**
- Iteración
  - Bucle For “clásico”
  - Bucle For-each
  - Bucle While
  - Tipo Iterator
- Ordenando colecciones de elementos
  - **Comparator**
  - Desordenando colecciones de elementos: **Shuffle**
- Notación **Lambda** y **Streams**
- Anexos
  - Objetos Anónimos
  - Arrays bidimensionales y bucles anidados
  - Utilidad de BlueJ: completar código

Notación Lambda y uso de Streams

- La notación *Lambda* y el uso de *Streams* aportan un uso distinto de *colecciones* e *iteración*.
- Complementan el estilo imperativo de programación utilizado hasta ahora con un estilo *funcional* de programación.
- Algunas librerías Java se han adaptado para el uso de expresiones *Lamda* y *Streams*

Notación *Lambda*

- Utiliza técnicas de lenguajes funcionales (Lisp, Haskell, etc.)
- Requiere una sintaxis adicional en el lenguaje

Streams

- Son una alternativa a la forma de iterar sobre colecciones
- Incluyen procesamiento de datos en varias etapas en forma de *pipeline* (flujo) de operaciones

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Notación Lambda

¿Qué es?

- Representación concisa (no escribimos código repetitivo) de funciones anónimas
- Pueden almacenarse en una variable o pasarse como parámetro a un método para ser ejecutada
- Tiene: un tipo de retorno, parámetros y un cuerpo (body)
- No es un método, no tiene nombre y no está asociada a ninguna clase u objeto

¿Cómo transformar un método en una expresión *Lambda*?

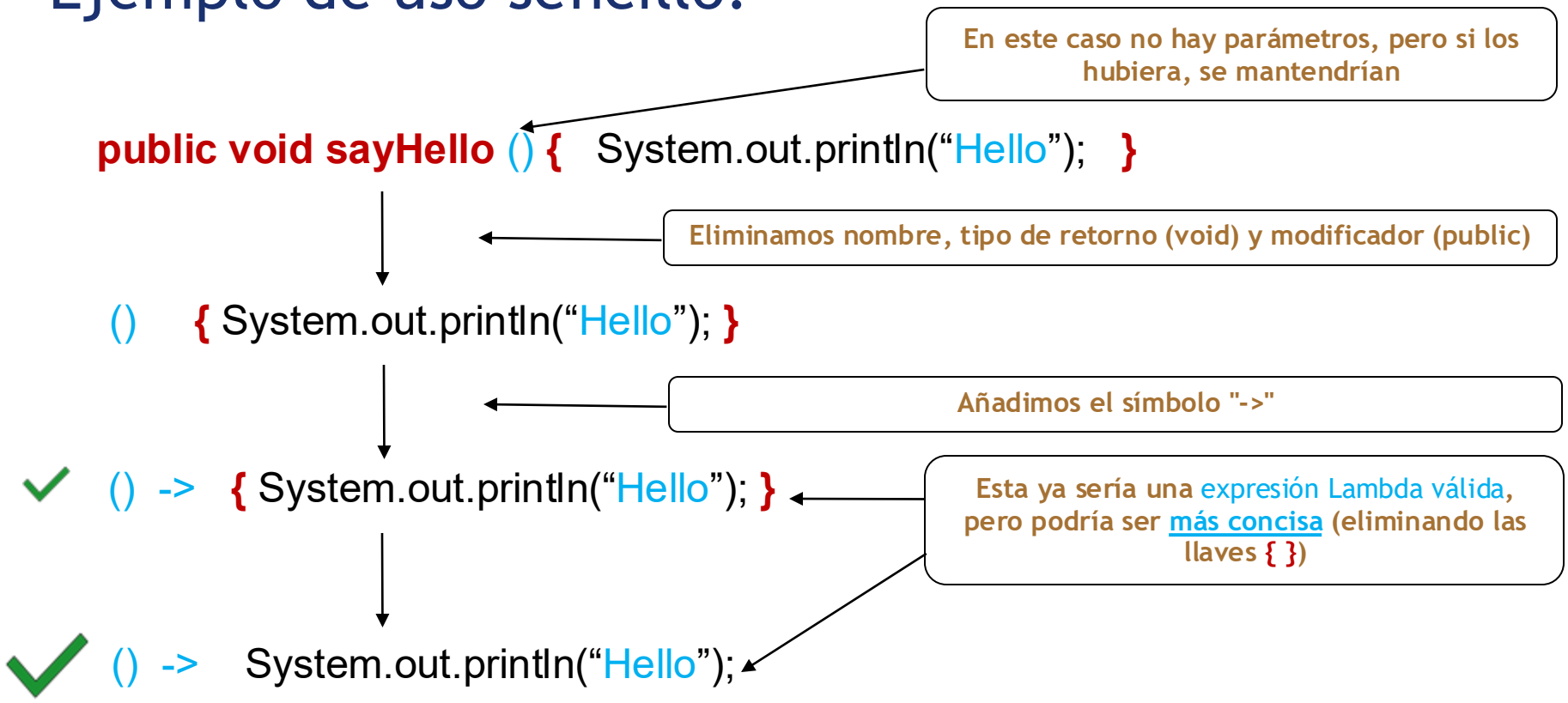
- Eliminar: el nombre, el tipo de retorno y el modificador (public, private, etc.)
- Añadir el símbolo flecha "->"

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Notación Lambda

Ejemplo de uso sencillo:



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Notación Lambda

Ejemplo de uso en colecciones:

Recorridos: Si quisiéramos recorrer una lista sería algo como:

```
bucle (for each elemento en la colección):
      obtener el elemento
      hacer algo con ese elemento
fin del bucle
```

```
for (Track pista: tracks)
{
    printTrack(pista);
}
```

```
public void printTrack(Track pista)
{
    System.out.println(pista.getDetails());
}
```

Cambiamos a notación Lambda

```
(Track pista)  ->
    System.out.println(pista.getDetails());
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Notación Lambda

Ejemplo de uso en colecciones:

Java 7:

```
for(String s: lista) {
    System.out.println(s);
}
```

Java 8:

```
collection.doThisForEachElement(some code);

lista.forEach(System.out::println);
```

Escribimos el código anterior en notación Lambda y Java 8:

```
for (Track elemento: tracks)
{
    printTrack(elemento);
}
```

Cambiamos a notación Lambda (en Java 8)

```
tracks.forEach (
(Track pista)  ->
        System.out.println(pista.getDetails());
);
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Streams

¿Qué son?

- Los Streams son un pipeline ("flujo") de funciones que se pueden evaluar
- Se suelen crear a partir del contenido de una colección. Por ejemplo, un ArrayList no es un stream, pero su método "stream" crea un stream de su contenido
- El acceso a los elementos de un stream no se acceden por un índice sino secuencialmente
- Los streams son inmutables, no se puede cambiar su orden o contenido, pero pueden cambiarse y almacenarse en una nueva estructura o colección
- Los elementos de un stream se pueden procesar en paralelo

```
List<Integer> evenNumbers = originalList.stream().filter(n -> n % 2 == 0).collect(Collectors.toList());
```

¿Qué obtendríamos en evenNumbers?

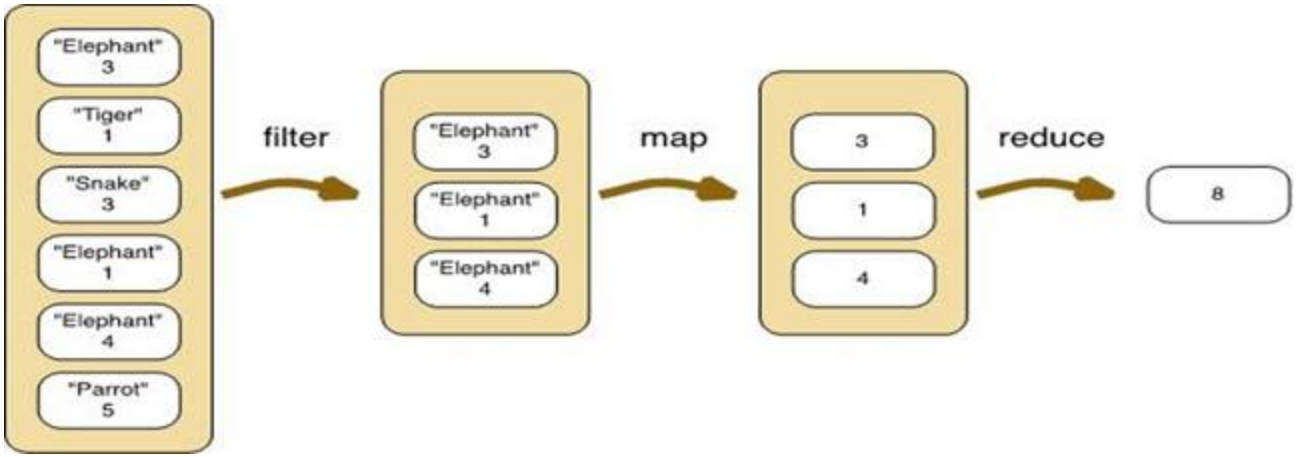
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Streams

Pipelines

- Comienzan con una fuente u origen
- Las operaciones pueden ser intermedias o finales
- Las operaciones intermedias producen un nuevo stream
- Las operaciones terminales son la operación final en el pipeline. Pueden tener un valor void como retorno




```
filter(name is elephant).map(count).reduce(add up)
```

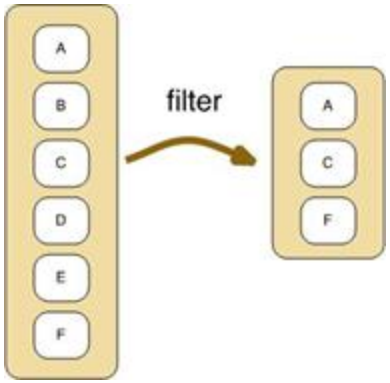
Pearson

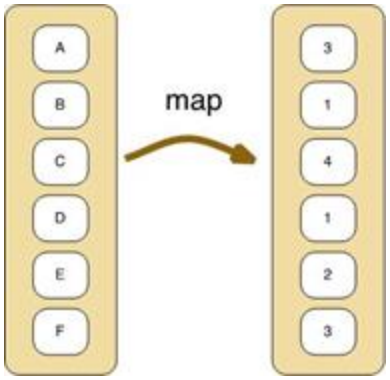
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Streams

Operaciones Intermedias

- Filter: devuelve una secuencia de elementos desde el stream de entrada para pasarlo al stream de salida
- Map: reemplaza elementos del stream de entrada con diferentes elementos en el stream de salida. Los elementos antes y después del mapeo pueden tener tipo distinto pero el número de elementos no cambia
- Distinct: devuelve un stream de elementos en el que cada elemento es único en el stream, se basa en el método equals



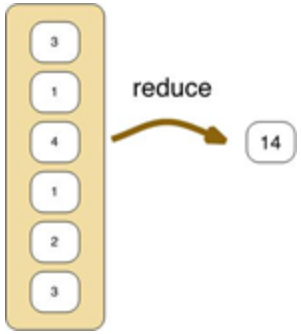


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Streams

Operaciones Finales



- Reduce: reduce los elementos del stream

  de entrada en un sólo elemento.

  Su salida es un tipo concreto de datos

- Collect: agrupa los elementos del stream de entrada . Su salida puede ser una colección de tipo List, Map o Set
- ForEach: realiza alguna acción en cada elemento del stream de entrada

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Streams Ejemplos

Mostrar lo detalles de las pistas (tracks) de David Bowie:

```
tracks.stream()
    .filter(pista -> "David Bowie".equals(pista.getArtist()))
    .forEach(pista -> System.out.println(pista.getDetails()));
```

Extraer las cadenas con los detalles de todas las pistas (tracks):

```
tracks.stream()
    .map(pista -> pista.getDetails())
    .forEach(details-> System.out.println(details));
```

método "map" (operación intermedia en un pipeline)

Agrupar las pistas (tracks) según su artista:

```
Map<String, List<Track>> tracksByArtist =
    tracks.stream()
    .collect(Collectors.groupingBy(pista-> pista.getArtist()));
```

Colección "Map" (pares de elementos <clave, valor>)

https://files.jrebel.com/pdf/zt_java8_streams_cheat_sheet.pdf

https://www.jrebel.com/blog/java-streams-cheat-sheet

https://dev.to/abh1navv/12-common-uses-of-java-streams-1pgk

https://www.arquitecturajava.com/java-stream-sorted-y-comparators/

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved



© 2003 United Feature Syndicate, Inc.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexos

Las diapositivas siguientes **deben** ser leídas **antes** de la siguiente sesión presencial y de laboratorio de la asignatura:
Unidad 04 – Anexo 01 – Objetos Anónimos
Unidad 04 – Anexo 02 – Arrays bidimensionales y bucles anidados
Unidad 04 – Anexo 03 – Utilidad de BlueJ: completar código

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 1

Objetos Anónimos

- Los objetos a menudo se crean de forma explícita y se usan en cualquier sitio inmediatamente:

```
Book otherBook = new Book(title,author);
myLibrary.add(otherBook);
```

- PERO, en el ejemplo anterior realmente no necesitamos `otherBook`:

```
myLibrary.add(new Book(title,author));
```

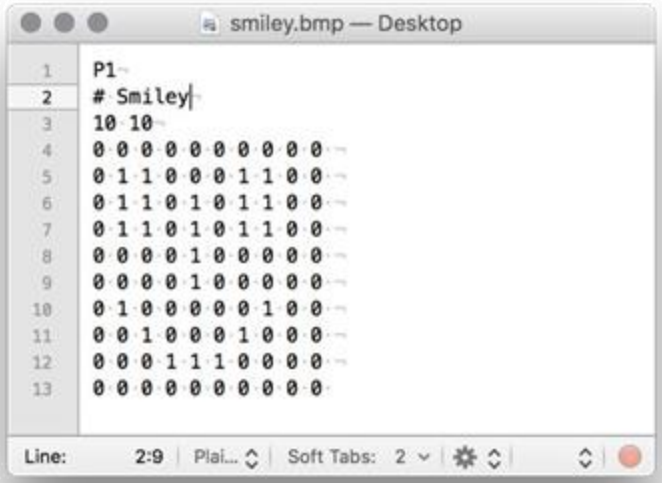
- Estamos creando un nuevo objeto **Book**.
- También estamos pasando el nuevo objeto al método **add** del **ArrayList** myLibrary

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Anexo 2
Arrays bidimensionales y bucles anidados

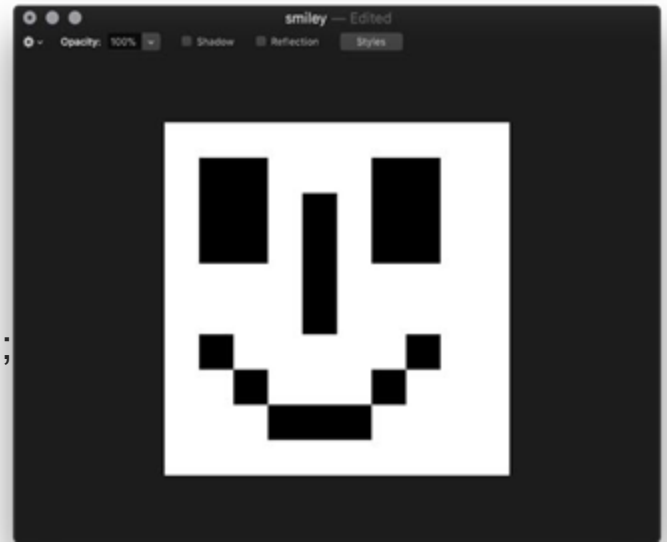
```
int[] row0 = {0,0,0,0,0,0,0,0,0,0};
int[] row1 = {0,1,1,0,0,0,1,1,0,0};
int[] row2 = {0,1,1,0,1,0,1,1,0,0};
int[] row3 = {0,1,1,0,1,0,1,1,0,0};
int[] row4 = {0,0,0,0,1,0,0,0,0,0};
int[] row5 = {0,0,0,0,1,0,0,0,0,0};
int[] row6 = {0,1,0,0,0,0,0,1,0,0};
int[] row7 = {0,0,1,0,0,0,1,0,0,0};
int[] row8 = {0,0,0,1,1,1,0,0,0,0};
int[] row9 = {0,0,0,0,0,0,0,0,0,0};
```




```
int[][] image = {row0, row1, row2, row3, row4, row5, row6, row7, row8, row9};
```

```
int w = 10;
int h = 10;

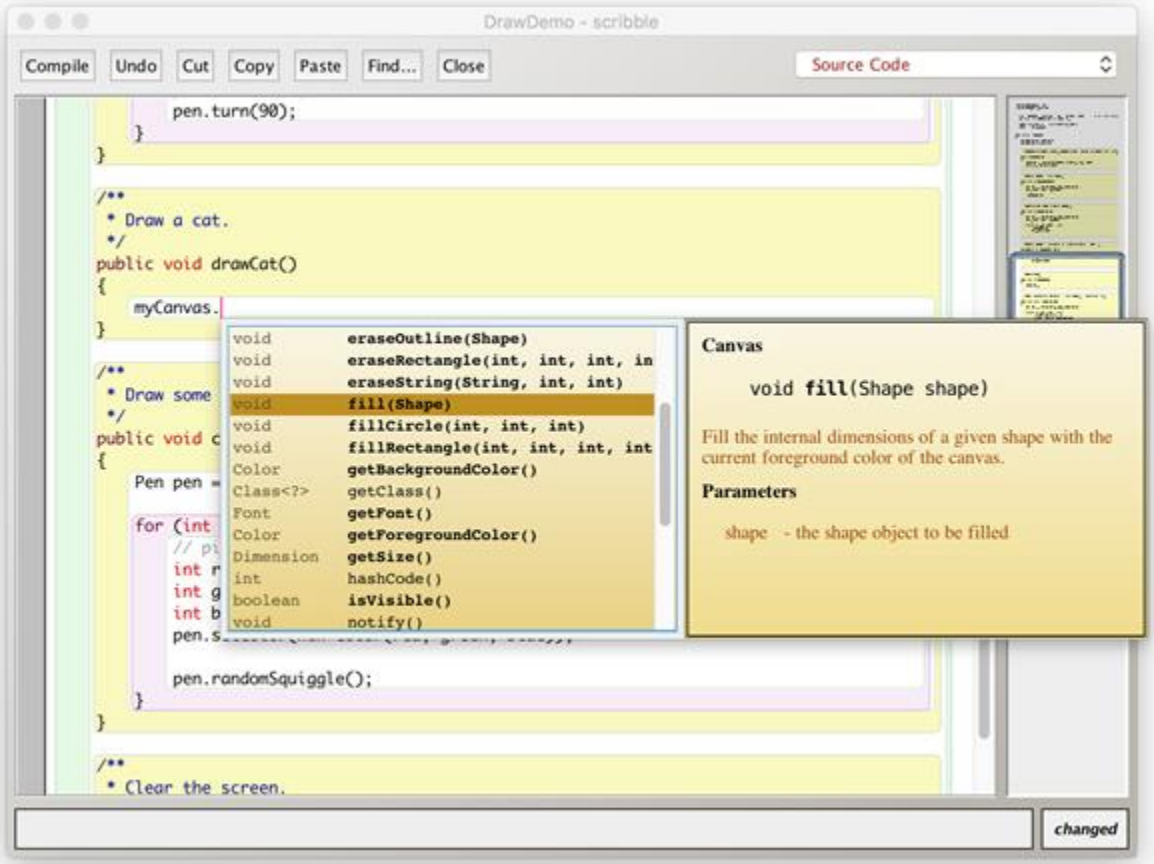
for (int y = 0; y < h; y++) {
    for (int x = 0; x < w; x++){
        System.out.print(image[y][x]+" ");
    }
    System.out.print("\n");
}
```



Anexo 3

Utilidad de BlueJ: completar código

- El editor de BlueJ soporta la búsqueda de métodos.
- Usamos ctrl-space detrás de la llamada a un método y el punto para que aparezca la lista de métodos disponibles.
- Usamos **Return para seleccionar** un método resaltado.



Ejercicios para pensar

Trata de resolverlos por tu cuenta

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 1 (1 de 2)

Dado el array numbers, muestra todos los números en el array usando el bucle for.

```
int[] numbers = { 4, 1, 22, 9, 14, 3, 9};

for ...
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 1 (2 de 2)

Dado el array numbers, muestra todos los números en el array usando el bucle for.

```
int[] numbers = { 4, 1, 22, 9, 14, 3, 9};

for(int num = 0; num < numbers.length; num++)
{
    System.out.println(numbers[num]);
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 2 (1 de 2)

Rellena un array con los primeros 10 números de la secuencia de Fibonacci.

```
0 1 1 2 3 5 8 13 21 34 ...
```

```
int[] fib = new int[10];

fib[0] = 0;
fib[1] = 1;

for(...) ...
```

on, Inc. All Rights Reserved

Ejercicio 2 (2 de 2)

Rellena un array con los primeros 10 números de la secuencia de Fibonacci.

```
int[] fib = new int[10];

fib[0] = 0;
fib[1] = 1;

for (int i = 2; i < fib.length; i++) {
    fib[i] = fib[i-2] + fib[i-1];
}
```

hts Reserved

Otra Bibliografía:

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Dr. Becka Morgan- Introduction to Computer Science. Presentations. University of Western Oregon
- Christiane Hagedorn, Ann Katrin Kuessner, Thomas Staubitz, Ralf Teusner. Object-Oriented Programming in Java. OpenSAP Courses

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved